

A3_housePrice

September 13, 2026

1 Assignment 3 - House Price Prediction

1.1 Intelligent System Development

This notebook follows the structure of the provided Assignment 3 / Deep Learning from Scratch material:

Data -> Understanding -> Cleaning -> Preprocessing -> Models -> Evaluation -> Comparison -> Conclusion

The deep learning model is implemented manually with NumPy. The neural network does not use TensorFlow, PyTorch, Keras, or any other deep learning framework.

1.1.1 Dataset

The dataset is the Singapore HDB resale flat price dataset provided for this assignment.

- Target: `resale_price`
- Problem type: regression
- Input: property, location, floor, lease, and transaction information
- File size: approximately 12.6 MB
- Raw rows: 134,168
- Raw columns: 11
- Source: <https://www.kaggle.com/datasets/mikelll/singapore-housing-price-20172022>

1.1.2 Four models to compare

1. Linear Regression
2. Random Forest Regressor
3. HistGradientBoosting Regressor
4. Deep Neural Network built from scratch with NumPy

The four models are evaluated on the same held-out test set. The comparison is performed **all four models together**, not as two separate comparison stages.

1.2 1. Learning Objectives

By the end of this notebook we should be able to:

1. Understand the structure and meaning of the house price dataset.
2. Detect and handle missing values and duplicate rows explicitly.
3. Inspect distributions, relationships, correlations, and potential outliers.

4. Build leakage-safe preprocessing from the training data.
5. Train three traditional machine learning regression models.
6. Explain a neural network as a composition of learned functions.
7. Implement forward propagation, MSE loss, backpropagation, and gradient descent manually.
8. Train a deep learning model without TensorFlow, PyTorch, or Keras.
9. Evaluate all four models with MAE, RMSE, R2, and MAPE.
10. Compare all four models with tables and multiple visualizations.

1.3 2. Why Is This a Regression Problem?

The target variable is `resale_price`, which is a continuous numerical value.

For example, a prediction may be:

Predicted price = 428000

This is different from classification, where the output is a class label such as 0 or 1.

Our goal is therefore:

Input features -> model -> predicted resale price

The main evaluation metrics will be:

- MAE: average absolute prediction error
- RMSE: square root of average squared error
- R2: proportion of variance explained by the model
- MAPE: average percentage error

For MAE, RMSE, and MAPE, lower values are better. For R2, higher values are better.

2 3. Imports and Reproducibility

This section imports the libraries used in the notebook.

Important restriction for the Deep Learning section:

- NumPy is used to implement the neural network.
- scikit-learn is used only for the three traditional ML models and their evaluation utilities.
- The neural network itself does not call TensorFlow, PyTorch, Keras, or a deep learning framework.

```
[1]: import os
import re
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor,
↳HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)

print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)

```

NumPy version: 2.4.6
Pandas version: 3.0.3

3 4. Locate and Load the Dataset

The expected folder structure is:

```

assignment 3/
  DATA/
    data/
      resale-flat-prices-based-on-registration-date-from-jan-2017-onwards.csv
    housePrice/
      housePrice_A3.ipynb

```

The following cell searches from the notebook location so the code remains portable.

```

[2]: notebook_dir = Path.cwd()
candidate_paths = [
    notebook_dir / "../data/"
    ↳resale-flat-prices-based-on-registration-date-from-jan-2017-onwards.csv",
    notebook_dir / "data/"
    ↳resale-flat-prices-based-on-registration-date-from-jan-2017-onwards.csv",
    Path("/mnt/data/assignment 3/DATA/data/"
    ↳resale-flat-prices-based-on-registration-date-from-jan-2017-onwards.csv"),
]

data_path = None
for p in candidate_paths:
    p = p.resolve()
    if p.exists():
        data_path = p
        break

if data_path is None:

```

```

raise FileNotFoundError(
    "CSV file was not found. Put the CSV inside assignment 3/DATA/data/."
)

df_raw = pd.read_csv(data_path)

print("Dataset path:", data_path)
print("Shape:", df_raw.shape)
print("File size (MB):", round(data_path.stat().st_size / (1024**2), 2))

```

Dataset path: C:\Users\ADMIN\Downloads\Intelligent System\assignment
 3\DATA\housePrice\data\resale-flat-prices-based-on-registration-date-from-
 jan-2017-onwards.csv
 Shape: (134168, 11)
 File size (MB): 12.59

```

[3]: print("First 5 rows:")
      display(df_raw.head())

      print("\nColumn names:")
      print(df_raw.columns.tolist())

      print("\nData types:")
      display(df_raw.dtypes)

```

First 5 rows:

	month	town	flat_type	block	street_name	storey_range	\
0	2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	
1	2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	
2	2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	
3	2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	
4	2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	

	floor_area_sqm	flat_model	lease_commence_date	remaining_lease	\
0	44.0	Improved	1979	61 years 04 months	
1	67.0	New Generation	1978	60 years 07 months	
2	67.0	New Generation	1980	62 years 05 months	
3	68.0	New Generation	1980	62 years 01 month	
4	67.0	New Generation	1980	62 years 05 months	

	resale_price
0	232000.0
1	250000.0
2	262000.0
3	265000.0
4	265000.0

Column names:

```
['month', 'town', 'flat_type', 'block', 'street_name', 'storey_range',  
'floor_area_sqm', 'flat_model', 'lease_commence_date', 'remaining_lease',  
'resale_price']
```

Data types:

```
month                str  
town                 str  
flat_type            str  
block                str  
street_name          str  
storey_range         str  
floor_area_sqm       float64  
flat_model           str  
lease_commence_date  int64  
remaining_lease      str  
resale_price         float64  
dtype: object
```

3.1 4.1 Initial Dataset Observations

The raw dataset contains property and transaction information such as:

- **month:** transaction month
- **town:** town / planning area
- **flat_type:** flat type
- **block:** block identifier
- **street_name:** street
- **storey_range:** storey interval
- **floor_area_sqm:** floor area
- **flat_model:** flat model
- **lease_commence_date:** lease start year
- **remaining_lease:** remaining lease as text
- **resale_price:** target value

At this stage we do not modify the data. We first inspect it, then make cleaning decisions based on evidence.

4 5. Data Quality Audit

A professional data-cleaning workflow should answer these questions explicitly:

1. Are there missing values?
2. Are there duplicate rows?
3. Are there incorrect data types?
4. Are there invalid values?
5. Are there extreme outliers?
6. Which columns require parsing or feature engineering?

The next cells perform each check separately.

```
[4]: print("Missing values per column:")
missing_summary = (
    df_raw.isna()
    .sum()
    .sort_values(ascending=False)
    .to_frame("missing_count")
)

missing_summary["missing_percent"] = (
    missing_summary["missing_count"] / len(df_raw) * 100
).round(3)

display(missing_summary)
```

Missing values per column:

	missing_count	missing_percent
month	0	0.0
town	0	0.0
flat_type	0	0.0
block	0	0.0
street_name	0	0.0
storey_range	0	0.0
floor_area_sqm	0	0.0
flat_model	0	0.0
lease_commence_date	0	0.0
remaining_lease	0	0.0
resale_price	0	0.0

4.0.1 Missing-value handling decision

The audit above should be read before any imputation.

For this exact dataset, the current uploaded CSV contains **0 missing values in every column**.

Therefore:

- No rows are removed because of missing values.
- No values are artificially filled in the current dataset.
- The notebook still contains a general imputation rule for robustness:
 - numeric columns -> median of the training data
 - categorical columns -> mode of the training data

Using the median for numeric data is less sensitive to extreme values than using the mean. Using the mode is a simple and interpretable rule for categorical data.

The important point is that imputation statistics are computed from the training data only after the train/test split.

```
[5]: duplicate_count = int(df_raw.duplicated().sum())

print("Exact duplicate rows:", duplicate_count)

if duplicate_count > 0:
    print("Action: remove exact duplicate rows before modeling.")
else:
    print("Action: no duplicate rows need to be removed.")
```

Exact duplicate rows: 259

Action: remove exact duplicate rows before modeling.

4.0.2 Duplicate handling decision

The raw dataset contains **259 exact duplicate rows**.

These rows are removed because they repeat the same complete observation. Keeping exact duplicates would give repeated records extra influence during model training.

This is different from removing similar houses. Only exact row-level duplicates are removed.

```
[6]: df = df_raw.copy()

# Remove exact duplicate observations.
df = df.drop_duplicates().reset_index(drop=True)

print("Rows before duplicate removal:", len(df_raw))
print("Rows after duplicate removal :", len(df))
print("Rows removed                :", len(df_raw) - len(df))
```

Rows before duplicate removal: 134168

Rows after duplicate removal : 133909

Rows removed : 259

```
[7]: print("Unique values per column:")
display(df.nunique().sort_values(ascending=False).to_frame("n_unique"))
```

Unique values per column:

	n_unique
resale_price	3221
block	2611
remaining_lease	653
street_name	559
floor_area_sqm	169
month	68
lease_commence_date	54
town	26
flat_model	21
storey_range	17
flat_type	7

5 6. Data Type Cleaning and Feature Engineering

Some useful information is stored as text and should be converted into numerical features before modeling.

5.0.1 Feature decisions

1. `month`
 - Parse into a real date.
 - Extract transaction year and transaction month.
2. `remaining_lease`
 - Convert strings such as 61 years 04 months into a numerical number of years.
3. `storey_range`
 - Convert ranges such as 10 TO 12 into the midpoint 11.
 - Keep the original categorical storey range as a separate categorical feature.
4. `block` and `street_name`
 - They are high-cardinality categorical variables.
 - Direct one-hot encoding would create many columns.
 - We later use training-set frequency encoding for these two columns.
5. `resale_price`
 - This is the target and is never used as an input feature.

```
[8]: def parse_remaining_lease(value):
    if pd.isna(value):
        return np.nan

    text = str(value).strip().lower()

    year_match = re.search(r"(\d+)\s+years?", text)
    month_match = re.search(r"(\d+)\s+months?", text)

    years = int(year_match.group(1)) if year_match else 0
    months = int(month_match.group(1)) if month_match else 0

    return years + months / 12.0

def parse_storey_mid(value):
    if pd.isna(value):
        return np.nan

    nums = re.findall(r"\d+", str(value))
    if len(nums) >= 2:
        return (int(nums[0]) + int(nums[1])) / 2.0
    if len(nums) == 1:
        return float(nums[0])

    return np.nan
```

```

df["month_date"] = pd.to_datetime(df["month"], format="%Y-%m", errors="coerce")
df["transaction_year"] = df["month_date"].dt.year
df["transaction_month"] = df["month_date"].dt.month
df["remaining_lease_years"] = df["remaining_lease"].apply(parse_remaining_lease)
df["storey_mid"] = df["storey_range"].apply(parse_storey_mid)

print("New engineered columns:")
print([
    "transaction_year",
    "transaction_month",
    "remaining_lease_years",
    "storey_mid"
])

display(
    df[
        [
            "month",
            "transaction_year",
            "transaction_month",
            "remaining_lease",
            "remaining_lease_years",
            "storey_range",
            "storey_mid"
        ]
    ].head()
)

```

New engineered columns:

['transaction_year', 'transaction_month', 'remaining_lease_years', 'storey_mid']

	month	transaction_year	transaction_month	remaining_lease \
0	2017-01	2017	1	61 years 04 months
1	2017-01	2017	1	60 years 07 months
2	2017-01	2017	1	62 years 05 months
3	2017-01	2017	1	62 years 01 month
4	2017-01	2017	1	62 years 05 months

	remaining_lease_years	storey_range	storey_mid
0	61.333333	10 TO 12	11.0
1	60.583333	01 TO 03	2.0
2	62.416667	01 TO 03	2.0
3	62.083333	04 TO 06	5.0
4	62.416667	01 TO 03	2.0

5.0.2 Invalid-value handling

Before modeling, we check logical constraints:

- `floor_area_sqm` should be greater than 0.
- `resale_price` should be greater than 0.
- `parsed_remaining_lease_years` should not be negative.
- `parsed_storey_mid` should not be negative.

Only records that violate a basic physical or mathematical constraint are removed.

We do **not** remove statistical outliers merely because they are large. Expensive flats are legitimate observations in a house-price problem, and automatically deleting them can remove meaningful market information.

```
[9]: invalid_mask = (
      (df["floor_area_sqm"] <= 0)
      | (df["resale_price"] <= 0)
      | (df["remaining_lease_years"] < 0)
      | (df["storey_mid"] < 0)
    )

    print("Invalid logical rows:", int(invalid_mask.sum()))

    df = df.loc[~invalid_mask].reset_index(drop=True)

    print("Shape after logical validation:", df.shape)
```

Invalid logical rows: 0

Shape after logical validation: (133909, 16)

```
[10]: # Final missing-value audit after parsing engineered features.
      missing_after_engineering = (
          df.isna()
          .sum()
          .sort_values(ascending=False)
          .to_frame("missing_count")
      )

      missing_after_engineering["missing_percent"] = (
          missing_after_engineering["missing_count"] / len(df) * 100
      ).round(3)

      display(missing_after_engineering[missing_after_engineering["missing_count"] > 0])
```

Empty DataFrame

Columns: [missing_count, missing_percent]

Index: []

5.0.3 Missing values after feature engineering

The original dataset has no missing values, but parsing text fields can create missing values if an unexpected format is encountered.

The audit cell above checks this explicitly.

If any missing value appears:

- numeric engineered/input feature -> median imputation
- categorical feature -> mode imputation

The imputation is implemented later using training-set statistics only. This avoids leaking information from the test set.

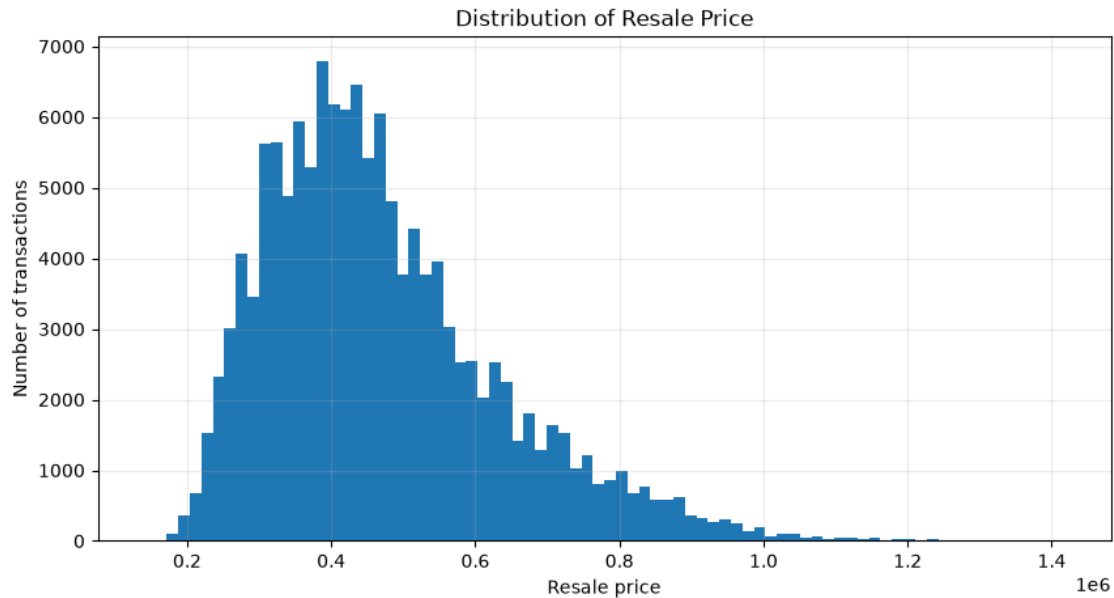
6 7. Exploratory Data Analysis

EDA is performed before modeling so that the model choices and preprocessing decisions are supported by evidence.

We will inspect:

1. Target distribution
2. Numerical feature distributions
3. Categorical distributions
4. Correlation between numerical variables
5. Price versus floor area
6. Price versus remaining lease
7. Price versus storey
8. Potential outliers using the IQR rule
9. Price trend over time

```
[11]: plt.figure(figsize=(10, 5))
plt.hist(df["resale_price"], bins=80)
plt.title("Distribution of Resale Price")
plt.xlabel("Resale price")
plt.ylabel("Number of transactions")
plt.grid(alpha=0.25)
plt.show()
```



6.0.1 Interpretation

House prices are usually not perfectly symmetric. A right-skewed target distribution is common because high-value flats can be much more expensive than the majority of transactions.

This matters because RMSE gives larger errors more weight. We therefore report both MAE and RMSE rather than using only one metric.

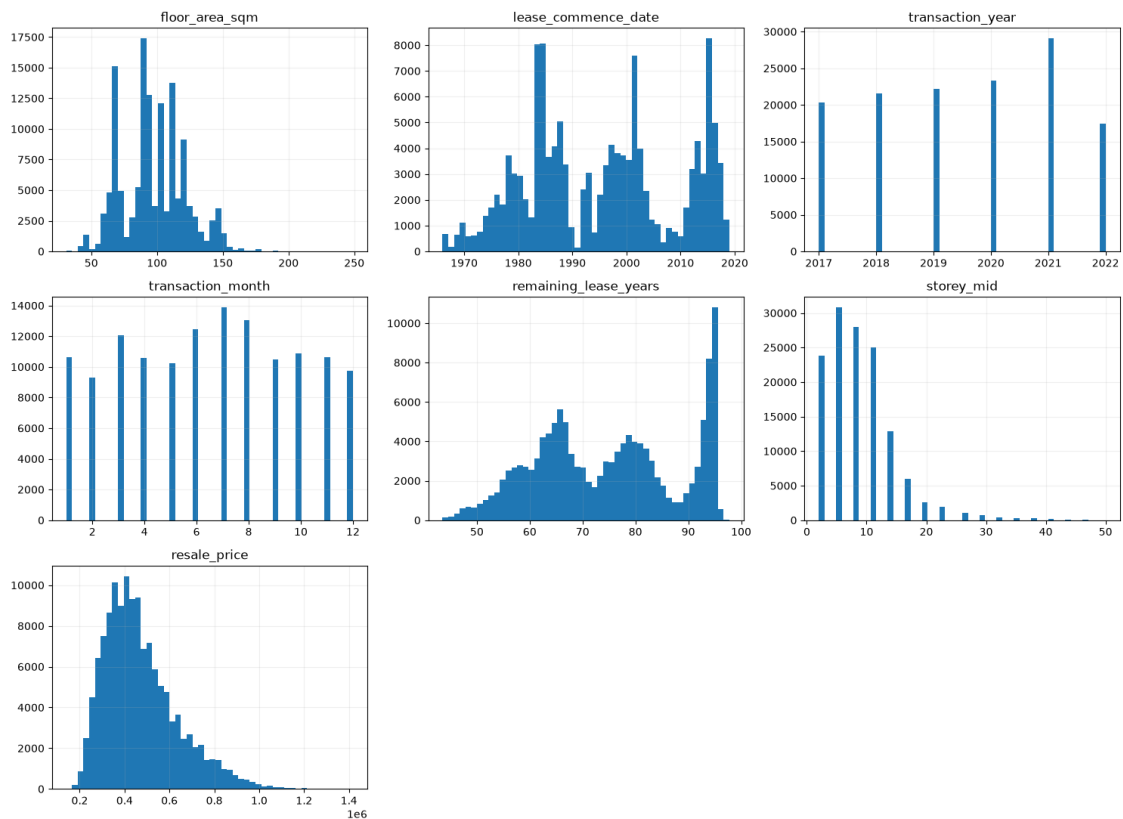
```
[12]: numeric_for_eda = [
    "floor_area_sqm",
    "lease_commence_date",
    "transaction_year",
    "transaction_month",
    "remaining_lease_years",
    "storey_mid",
    "resale_price",
]

fig, axes = plt.subplots(3, 3, figsize=(15, 11))
axes = axes.ravel()

for i, col in enumerate(numeric_for_eda):
    axes[i].hist(df[col].dropna(), bins=50)
    axes[i].set_title(col)
    axes[i].grid(alpha=0.2)

for j in range(len(numeric_for_eda), len(axes)):
    axes[j].axis("off")
```

```
plt.tight_layout()
plt.show()
```

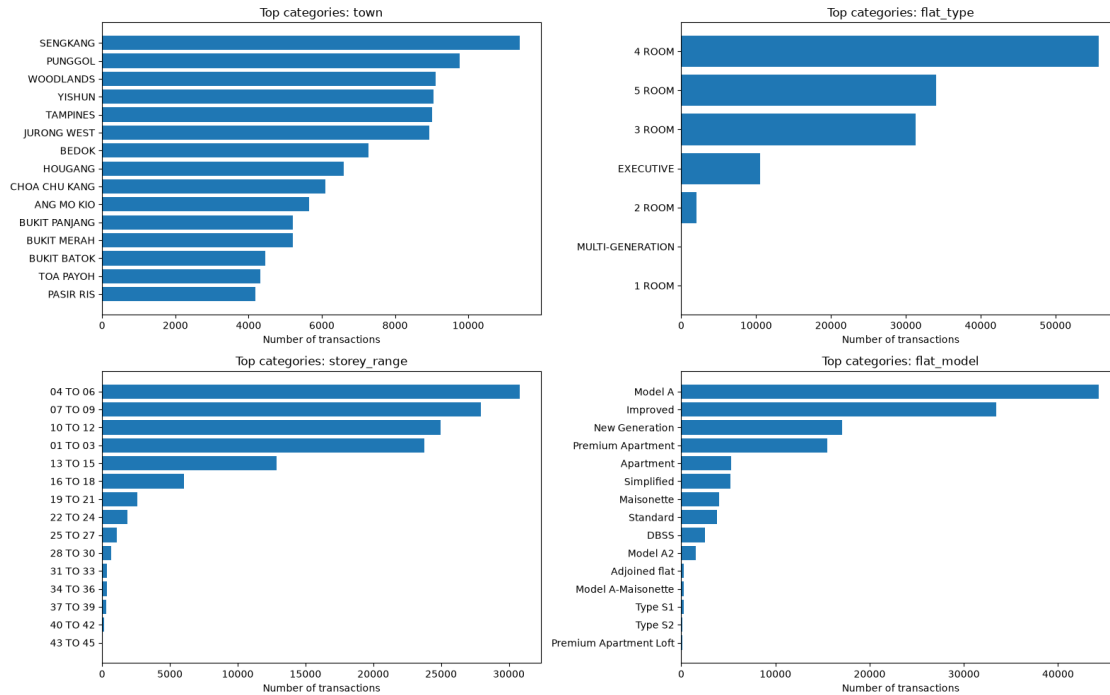


```
[13]: categorical_columns = ["town", "flat_type", "storey_range", "flat_model"]

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.ravel()

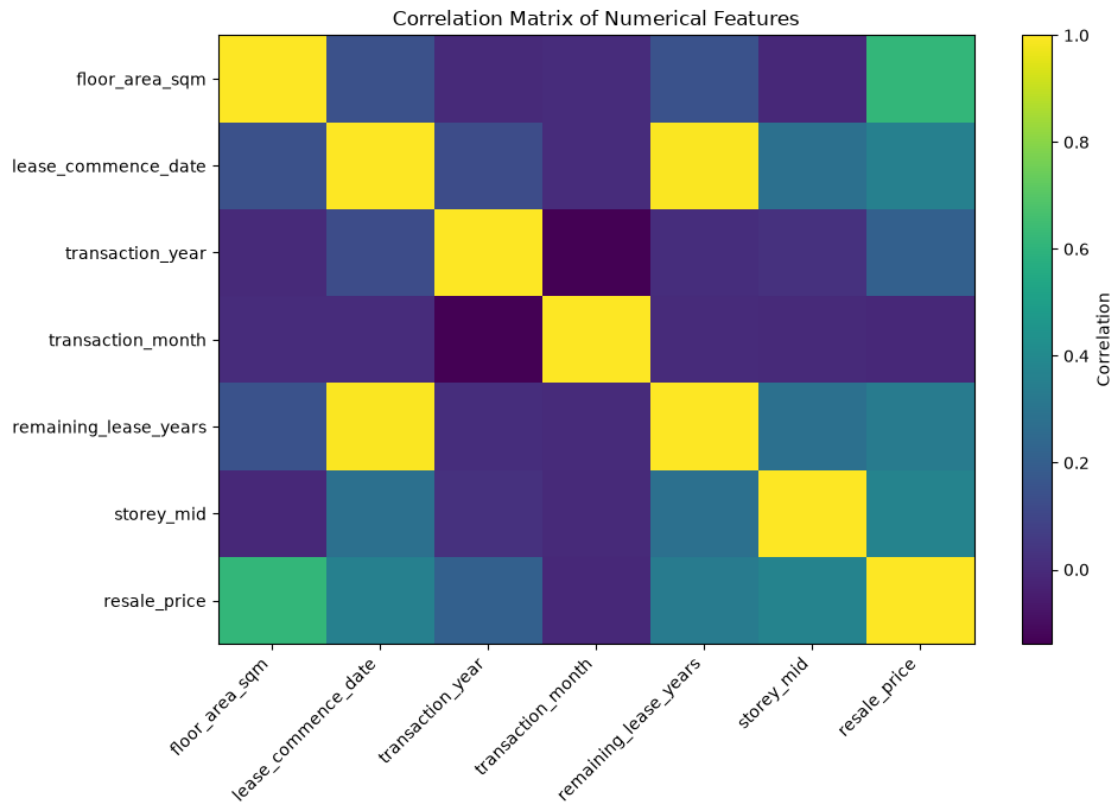
for ax, col in zip(axes, categorical_columns):
    counts = df[col].value_counts().head(15).sort_values()
    axes[list(axes).index(ax)].barh(counts.index, counts.values)
    ax.set_title(f"Top categories: {col}")
    ax.set_xlabel("Number of transactions")

plt.tight_layout()
plt.show()
```

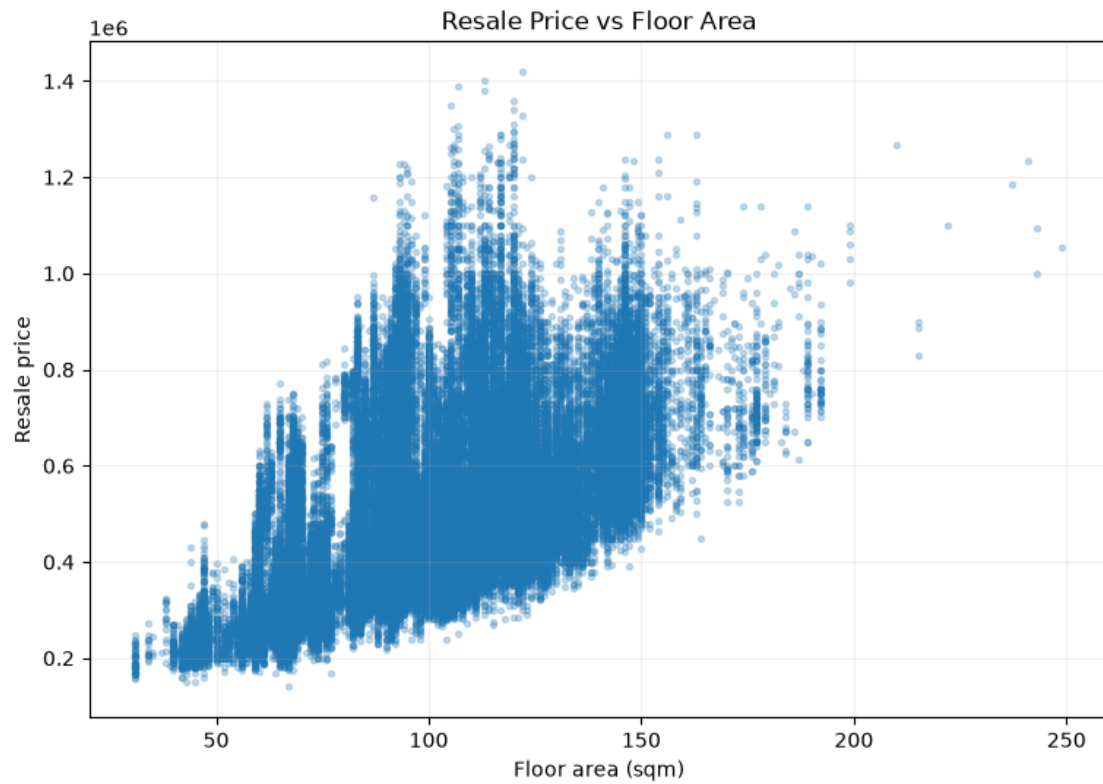


```
[14]: corr = df[numeric_for_eda].corr(numeric_only=True)

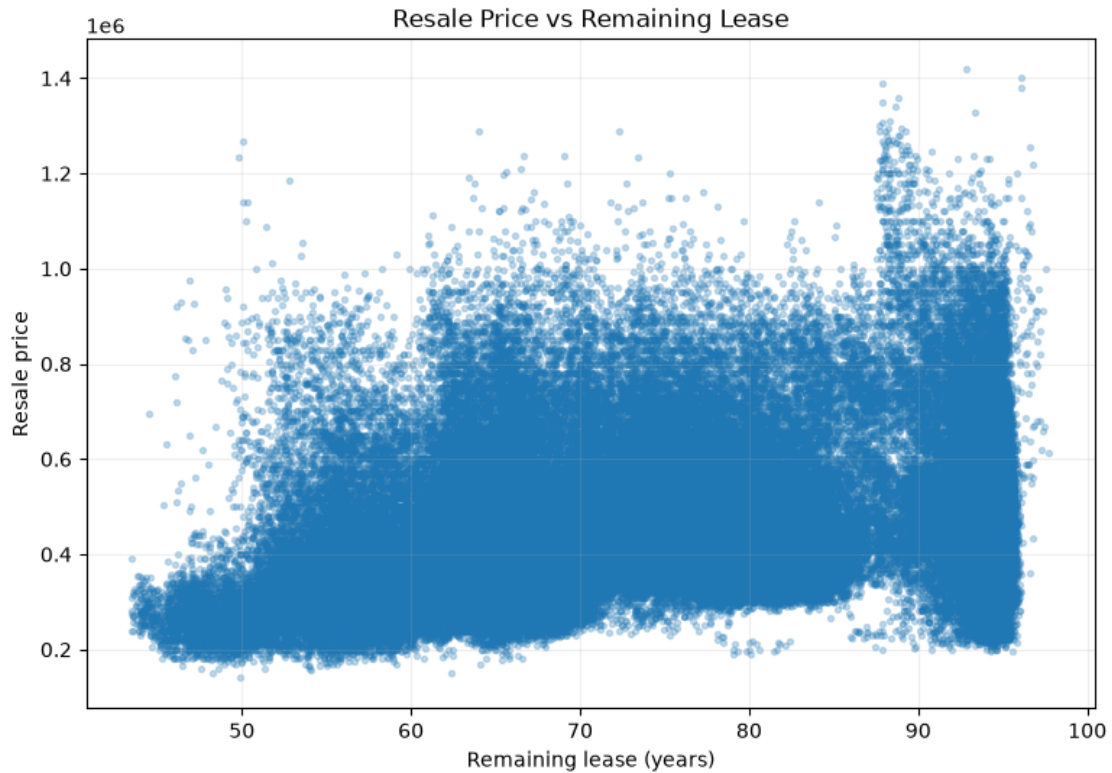
plt.figure(figsize=(10, 7))
plt.imshow(corr, aspect="auto")
plt.colorbar(label="Correlation")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr.index)), corr.index)
plt.title("Correlation Matrix of Numerical Features")
plt.tight_layout()
plt.show()
```



```
[15]: plt.figure(figsize=(9, 6))
plt.scatter(
    df["floor_area_sqm"],
    df["resale_price"],
    s=8,
    alpha=0.25
)
plt.title("Resale Price vs Floor Area")
plt.xlabel("Floor area (sqm)")
plt.ylabel("Resale price")
plt.grid(alpha=0.2)
plt.show()
```

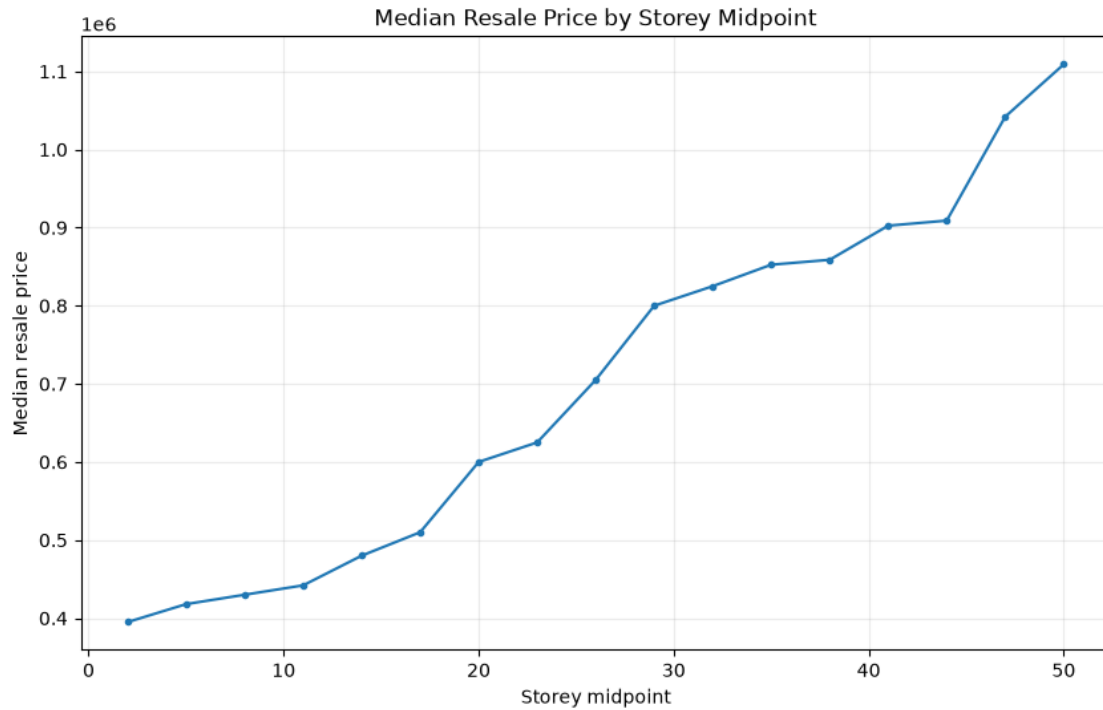


```
[16]: plt.figure(figsize=(9, 6))
plt.scatter(
    df["remaining_lease_years"],
    df["resale_price"],
    s=8,
    alpha=0.25
)
plt.title("Resale Price vs Remaining Lease")
plt.xlabel("Remaining lease (years)")
plt.ylabel("Resale price")
plt.grid(alpha=0.2)
plt.show()
```

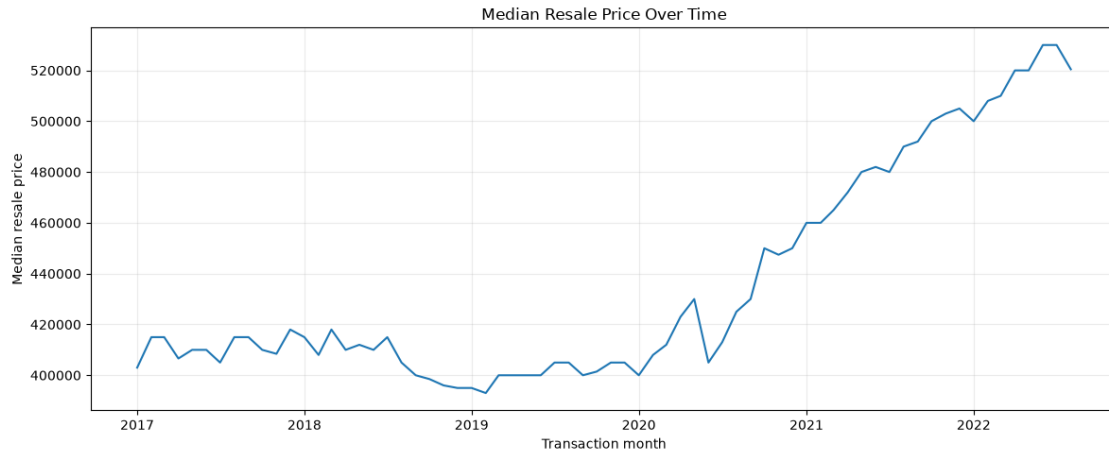


```
[17]: plt.figure(figsize=(10, 6))
storey_price = (
    df.groupby("storey_mid")["resale_price"]
    .median()
    .sort_index()
)

plt.plot(storey_price.index, storey_price.values, marker="o", markersize=3)
plt.title("Median Resale Price by Storey Midpoint")
plt.xlabel("Storey midpoint")
plt.ylabel("Median resale price")
plt.grid(alpha=0.25)
plt.show()
```



```
[18]: monthly_price = (  
    df.groupby("month_date")["resale_price"]  
    .median()  
    .sort_index()  
)  
  
plt.figure(figsize=(12, 5))  
plt.plot(monthly_price.index, monthly_price.values)  
plt.title("Median Resale Price Over Time")  
plt.xlabel("Transaction month")  
plt.ylabel("Median resale price")  
plt.grid(alpha=0.25)  
plt.tight_layout()  
plt.show()
```



7 8. Outlier Analysis

We use the IQR rule only as a diagnostic:

$$\text{IQR} = Q3 - Q1$$

$$\text{Lower bound} = Q1 - 1.5 * \text{IQR}$$

$$\text{Upper bound} = Q3 + 1.5 * \text{IQR}$$

An observation outside these bounds is statistically unusual, but not automatically wrong.

For this dataset, we keep valid price and property observations even when they are statistical outliers. The reason is that the goal is to predict real resale prices, including legitimate expensive properties.

```
[19]: def iqr_outlier_count(series):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    count = int(((series < lower) | (series > upper)).sum())
    return count, lower, upper

outlier_report = []

for col in [
    "floor_area_sqm",
    "remaining_lease_years",
    "storey_mid",
    "resale_price",
]:
    count, lower, upper = iqr_outlier_count(df[col].dropna())
```

```

outlier_report.append({
    "feature": col,
    "iqr_outlier_count": count,
    "lower_bound": lower,
    "upper_bound": upper,
})

display(pd.DataFrame(outlier_report))

```

	feature	iqr_outlier_count	lower_bound	upper_bound
0	floor_area_sqm	772	35.500	159.500000
1	remaining_lease_years	0	31.125	118.791667
2	storey_mid	4925	-4.000	20.000000
3	resale_price	3521	42500.000	862500.000000

8 9. Prepare the Modeling Table

We now separate:

- target: `resale_price`
- input features

The raw text column `month` is replaced by its parsed year and month. The raw text column `remaining_lease` is replaced by the numerical `remaining_lease_years` representation. The original `month_date` is also excluded from the model because it is only needed for EDA.

8.0.1 Encoding strategy

Low-cardinality categorical features:

- `town`
- `flat_type`
- `storey_range`
- `flat_model`

These are one-hot encoded.

High-cardinality categorical features:

- `block`
- `street_name`

These use **frequency encoding**:

```
frequency(category) = count(category) / number_of_training_rows
```

The frequency map is learned from the training split only. This prevents test-set information from influencing preprocessing.

```

[20]: target_col = "resale_price"

categorical_low = [

```

```

    "town",
    "flat_type",
    "storey_range",
    "flat_model",
]

categorical_high = [
    "block",
    "street_name",
]

numeric_features = [
    "floor_area_sqm",
    "lease_commence_date",
    "transaction_year",
    "transaction_month",
    "remaining_lease_years",
    "storey_mid",
]

feature_columns = (
    categorical_low
    + categorical_high
    + numeric_features
)

X = df[feature_columns].copy()
y = df[target_col].copy()

print("X shape:", X.shape)
print("y shape:", y.shape)
display(X.head())

```

X shape: (133909, 12)

y shape: (133909,)

	town	flat_type	storey_range	flat_model	block	street_name	\
0	ANG MO KIO	2 ROOM	10 TO 12	Improved	406	ANG MO KIO AVE 10	
1	ANG MO KIO	3 ROOM	01 TO 03	New Generation	108	ANG MO KIO AVE 4	
2	ANG MO KIO	3 ROOM	01 TO 03	New Generation	602	ANG MO KIO AVE 5	
3	ANG MO KIO	3 ROOM	04 TO 06	New Generation	465	ANG MO KIO AVE 10	
4	ANG MO KIO	3 ROOM	01 TO 03	New Generation	601	ANG MO KIO AVE 5	

	floor_area_sqm	lease_commence_date	transaction_year	transaction_month	\
0	44.0	1979	2017	1	
1	67.0	1978	2017	1	
2	67.0	1980	2017	1	
3	68.0	1980	2017	1	

4	67.0	1980	2017	1
---	------	------	------	---

	remaining_lease_years	storey_mid
0	61.333333	11.0
1	60.583333	2.0
2	62.416667	2.0
3	62.083333	5.0
4	62.416667	2.0

9 10. Train/Test Split

We use an 80/20 random split with a fixed seed so that:

- 80% of the observations are used for training.
- 20% are held out for final testing.
- the same split is used by all four models.
- the comparison is therefore fair.

The preprocessing statistics are learned from the training set only.

```
[21]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=SEED,
)

print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))
```

```
Training samples: 107127
Testing samples : 26782
```

10 11. Leakage-Safe Preprocessing

This cell implements the preprocessing without relying on a black-box preprocessing pipeline, so that every step is visible.

10.0.1 Numeric features

Missing values are filled with the training median.

10.0.2 Categorical low-cardinality features

Missing values are filled with the training mode, then one-hot encoded.

10.0.3 High-cardinality features

block and street_name are replaced with their training-set frequencies.

10.0.4 Scaling

Only continuous numerical features are standardized:

$$z = (x - \text{mean}) / \text{std}$$

The mean and standard deviation are calculated from `X_train` only.

The same training statistics are then applied to `X_test`.

This is exactly the same principle shown in the A3 Deep Learning example: the test set must use training-set normalization parameters.

```
[22]: def fit_frequency_map(series):
    freq = series.value_counts(dropna=False) / len(series)
    return freq.to_dict()

def apply_frequency_encoding(series, mapping):
    # Unknown test categories receive frequency 0.
    return series.map(mapping).fillna(0.0)

def fit_transform_features(X_train, X_test):
    train = X_train.copy()
    test = X_test.copy()

    # 1. Numeric imputation using training medians.
    train_medians = train[numeric_features].median()

    train[numeric_features] = train[numeric_features].fillna(train_medians)
    test[numeric_features] = test[numeric_features].fillna(train_medians)

    # 2. High-cardinality categorical features -> frequency encoding.
    frequency_maps = {}

    for col in categorical_high:
        train[col] = train[col].astype(str)
        test[col] = test[col].astype(str)

        freq_map = fit_frequency_map(train[col])
        frequency_maps[col] = freq_map

        train[col + "_freq"] = apply_frequency_encoding(train[col], freq_map)
        test[col + "_freq"] = apply_frequency_encoding(test[col], freq_map)

    # 3. Low-cardinality categoricals -> training-based modes + one-hot
    ↪ encoding.
    train_modes = {}
    for col in categorical_low:
```

```

mode = train[col].mode(dropna=True)
mode_value = mode.iloc[0] if len(mode) else "UNKNOWN"
train_modes[col] = mode_value
train[col] = train[col].fillna(mode_value).astype(str)
test[col] = test[col].fillna(mode_value).astype(str)

train_dummies = pd.get_dummies(
    train[categorical_low],
    prefix=categorical_low,
    dtype=float
)

test_dummies = pd.get_dummies(
    test[categorical_low],
    prefix=categorical_low,
    dtype=float
)

# Make test columns exactly match training columns.
test_dummies = test_dummies.reindex(
    columns=train_dummies.columns,
    fill_value=0.0
)

# 4. Build the continuous numerical part.
continuous_cols = numeric_features + [
    col + "_freq" for col in categorical_high
]

train_cont = pd.concat(
    [
        train[numeric_features].astype(float),
        train[[col + "_freq" for col in categorical_high]].astype(float),
    ],
    axis=1
)

test_cont = pd.concat(
    [
        test[numeric_features].astype(float),
        test[[col + "_freq" for col in categorical_high]].astype(float),
    ],
    axis=1
)

# 5. Standardize continuous columns using only training statistics.
means = train_cont.mean()

```

```

stds = train_cont.std().replace(0, 1.0)

train_cont = (train_cont - means) / stds
test_cont = (test_cont - means) / stds

# 6. Combine continuous and one-hot features.
train_final = pd.concat(
    [train_cont.reset_index(drop=True), train_dummies.
↪reset_index(drop=True)],
    axis=1
)

test_final = pd.concat(
    [test_cont.reset_index(drop=True), test_dummies.reset_index(drop=True)],
    axis=1
)

preprocessing_info = {
    "numeric_medians": train_medians,
    "categorical_modes": train_modes,
    "frequency_maps": frequency_maps,
    "continuous_means": means,
    "continuous_stds": stds,
    "feature_names": train_final.columns.tolist(),
    "continuous_columns": continuous_cols,
    "dummy_columns": train_dummies.columns.tolist(),
}

return (
    train_final.astype(np.float32),
    test_final.astype(np.float32),
    preprocessing_info,
)

X_train_processed, X_test_processed, preprocessing_info = ↪
↪fit_transform_features(
    X_train,
    X_test
)

print("Processed training shape:", X_train_processed.shape)
print("Processed testing shape :", X_test_processed.shape)
print("Number of final features:", X_train_processed.shape[1])

```

```

Processed training shape: (107127, 79)
Processed testing shape : (26782, 79)

```

Number of final features: 79

10.0.5 Preprocessing verification

The processed matrices must satisfy:

- same number of columns
- no missing values
- no infinite values
- test data transformed using the training statistics

```
[23]: Xtr = X_train_processed.to_numpy(dtype=np.float64)
      Xte = X_test_processed.to_numpy(dtype=np.float64)

      print("Same feature count:", Xtr.shape[1] == Xte.shape[1])
      print("Training NaNs:", np.isnan(Xtr).sum())
      print("Testing NaNs :", np.isnan(Xte).sum())
      print("Training infs:", np.isinf(Xtr).sum())
      print("Testing infs :", np.isinf(Xte).sum())
```

Same feature count: True

Training NaNs: 0

Testing NaNs : 0

Training infs: 0

Testing infs : 0

11 12. Model 1 - Linear Regression

Linear Regression provides a strong and interpretable baseline.

Conceptually:

$$\hat{y} = XW + b$$

The model assumes that the target can be approximated as a weighted combination of the input features.

It is useful as a baseline because more complex models should demonstrate a meaningful improvement over this simpler mapping.

```
[24]: y_train_np = y_train.to_numpy(dtype=np.float64)
      y_test_np = y_test.to_numpy(dtype=np.float64)

      model_lr = LinearRegression()

      start = time.time()
      model_lr.fit(Xtr, y_train_np)
      lr_train_time = time.time() - start

      pred_lr = model_lr.predict(Xte)
```

```
print("Linear Regression training time:", round(lr_train_time, 2), "seconds")
```

Linear Regression training time: 0.23 seconds

12 13. Model 2 - Random Forest Regressor

Random Forest builds many decision trees and averages their predictions.

Main idea:

Input -> many decision trees -> average predictions

It can capture nonlinear relationships that a linear model cannot represent.

The model is trained on exactly the same processed training matrix and evaluated on exactly the same test matrix.

```
[25]: model_rf = RandomForestRegressor(  
        n_estimators=160,  
        max_depth=20,  
        min_samples_leaf=2,  
        random_state=SEED,  
        n_jobs=-1,  
    )  
  
    start = time.time()  
    model_rf.fit(Xtr, y_train_np)  
    rf_train_time = time.time() - start  
  
    pred_rf = model_rf.predict(Xte)  
  
    print("Random Forest training time:", round(rf_train_time, 2), "seconds")
```

Random Forest training time: 23.96 seconds

13 14. Model 3 - HistGradientBoosting Regressor

HistGradientBoosting builds trees sequentially.

Each new tree focuses on reducing the errors made by the current ensemble.

Conceptually:

Model 1 -> residual errors -> Model 2 -> residual errors -> ...

This is a boosting approach and is useful for comparing a sequential ensemble with Random Forest.

```
[26]: model_hgb = HistGradientBoostingRegressor(  
        max_iter=300,  
        learning_rate=0.08,  
        max_leaf_nodes=31,
```

```

    l2_regularization=0.1,
    random_state=SEED,
)

start = time.time()
model_hgb.fit(Xtr, y_train_np)
hgb_train_time = time.time() - start

pred_hgb = model_hgb.predict(Xte)

print(
    "HistGradientBoosting training time:",
    round(hgb_train_time, 2),
    "seconds"
)

```

HistGradientBoosting training time: 7.79 seconds

14 15. Evaluate the Three Traditional ML Models

We now evaluate the models with the same metrics.

14.0.1 MAE

```
MAE = mean(abs(y - y_hat))
```

Interpretation: average absolute prediction error.

14.0.2 RMSE

```
RMSE = sqrt(mean((y - y_hat)^2))
```

RMSE penalizes large errors more strongly than MAE.

14.0.3 R2

```
R2 = 1 - residual_sum_of_squares / total_sum_of_squares
```

Higher is better.

14.0.4 MAPE

```
MAPE = mean(abs((y - y_hat) / y)) * 100
```

Lower is better.

MAPE must be interpreted carefully when true target values are near zero. In this dataset resale prices are positive and far from zero, so it is usable as an additional metric.

```
[27]: def regression_metrics(y_true, y_pred):
      mae = mean_absolute_error(y_true, y_pred)
```

```

rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2 = r2_score(y_true, y_pred)

y_true_arr = np.asarray(y_true, dtype=float)
y_pred_arr = np.asarray(y_pred, dtype=float)
mape = np.mean(
    np.abs((y_true_arr - y_pred_arr) / np.maximum(np.abs(y_true_arr), 1e-8))
) * 100

return {
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    "MAPE (%)": mape,
}

traditional_results = pd.DataFrame(
    [
        {"Model": "Linear Regression", **regression_metrics(y_test, pred_lr)},
        {"Model": "Random Forest", **regression_metrics(y_test, pred_rf)},
        {"Model": "HistGradientBoosting", **regression_metrics(y_test,
↪pred_hgb)},
    ]
)

display(traditional_results.round(3))

```

	Model	MAE	RMSE	R2	MAPE (%)
0	Linear Regression	47471.503	60873.799	0.859	10.817
1	Random Forest	26654.031	37886.863	0.945	5.784
2	HistGradientBoosting	23262.656	31489.940	0.962	5.065

15 16. Traditional ML Visualization

The comparison should not rely only on a table.

We create:

1. Metric comparison bar charts
2. Actual versus predicted scatter plots
3. Residual distributions

A good regression model should place points close to the diagonal line in actual-versus-predicted plots.

```

[28]: metrics_to_plot = ["MAE", "RMSE", "R2", "MAPE (%)"]

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

```

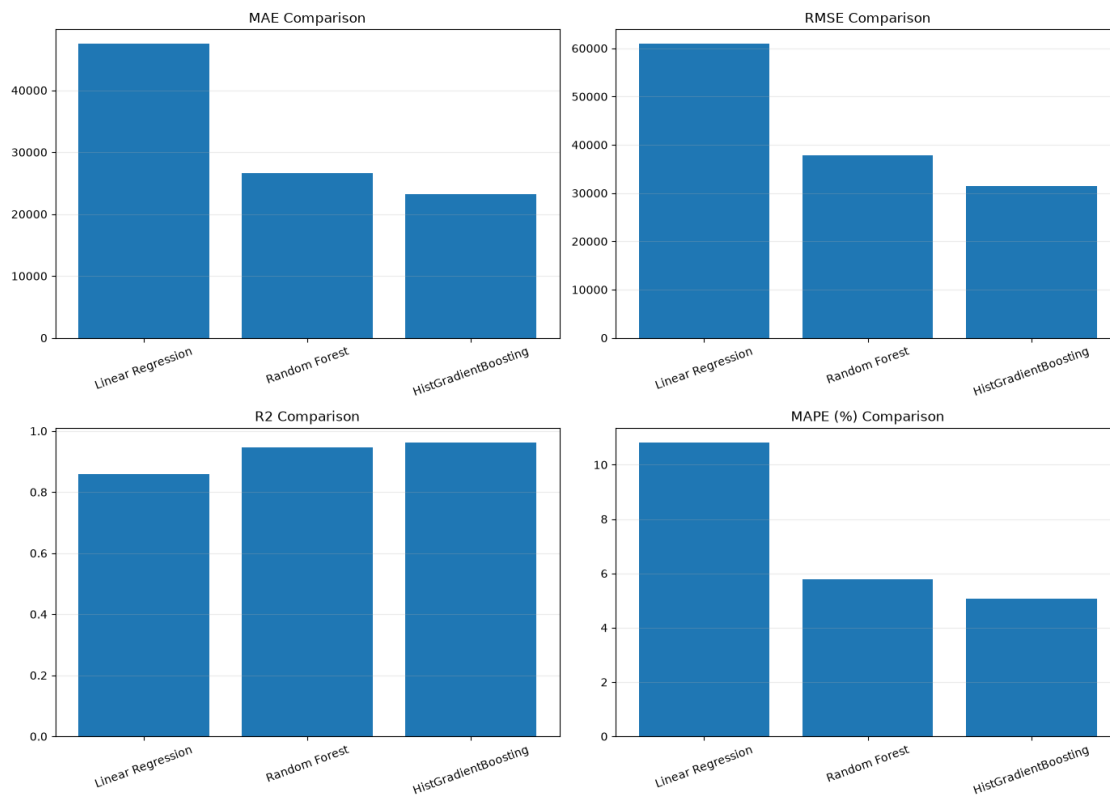
```

axes = axes.ravel()

for ax, metric in zip(axes, metrics_to_plot):
    ax.bar(traditional_results["Model"], traditional_results[metric])
    ax.set_title(f"{metric} Comparison")
    ax.tick_params(axis="x", rotation=20)
    ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
plt.show()

```



```

[29]: predictions_for_plot = {
        "Linear Regression": pred_lr,
        "Random Forest": pred_rf,
        "HistGradientBoosting": pred_hgb,
    }

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

y_min = min(y_test.min(), pred_lr.min(), pred_rf.min(), pred_hgb.min())
y_max = max(y_test.max(), pred_lr.max(), pred_rf.max(), pred_hgb.max())

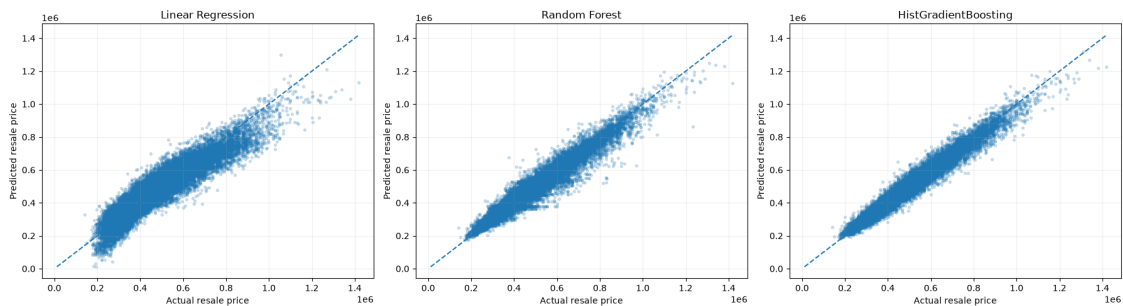
```

```

for ax, (name, pred) in zip(axes, predictions_for_plot.items()):
    ax.scatter(y_test, pred, s=8, alpha=0.2)
    ax.plot([y_min, y_max], [y_min, y_max], linestyle="--")
    ax.set_title(name)
    ax.set_xlabel("Actual resale price")
    ax.set_ylabel("Predicted resale price")
    ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()

```



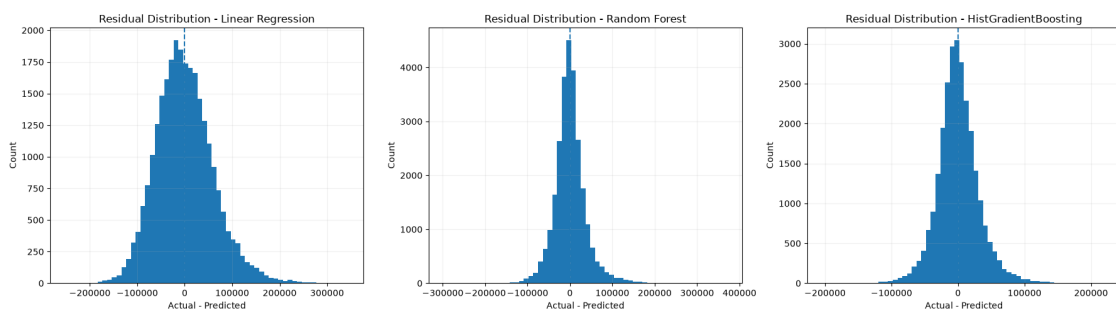
```

[30]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, pred) in zip(axes, predictions_for_plot.items()):
    residuals = np.asarray(y_test) - pred
    ax.hist(residuals, bins=60)
    ax.axvline(0, linestyle="--")
    ax.set_title(f"Residual Distribution - {name}")
    ax.set_xlabel("Actual - Predicted")
    ax.set_ylabel("Count")
    ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()

```



16 17. Deep Learning from Scratch - Concept

The provided A3 material emphasizes the central idea:

Deep Learning = composition of learned functions

For HousePrice, we use a regression network:

Input features

|

v

Dense Layer 1: input_dim -> 32

|

ReLU

|

v

Dense Layer 2: 32 -> 16

|

ReLU

|

v

Output Layer: 16 -> 1

|

Linear output

|

v

Predicted price

This follows the same conceptual pattern as the A3 diabetes example:

$X \rightarrow H1 \rightarrow H2 \rightarrow \hat{y}$

The important difference is the task:

- Diabetes classification -> sigmoid output + binary cross-entropy
- HousePrice regression -> linear output + mean squared error

17 18. Deep Learning Mathematics

For the first layer:

$$Z1 = XW1 + b1$$

$$H1 = \text{ReLU}(Z1)$$

For the second layer:

$$Z2 = H1W2 + b2$$

$$H2 = \text{ReLU}(Z2)$$

For the output:

```
Z3 = H2W3 + b3  
y_hat = Z3
```

Because this is regression, we do not apply sigmoid to the final output.

17.0.1 ReLU

$\text{ReLU}(x) = \max(0, x)$

17.0.2 Mean Squared Error

$\text{MSE} = (1/N) * \sum((y_hat - y)^2)$

17.0.3 Learning objective

We want model parameters that minimize MSE.

The complete learning cycle remains:

Forward -> Loss -> Backpropagation -> Gradient Descent

18 19. Prepare the Data for the NumPy Neural Network

The neural network below uses NumPy directly.

No sklearn model or sklearn neural-network implementation is used.

Because house prices are large values, the target is standardized for neural-network training:

```
y_scaled = (y - mean_train) / std_train
```

After prediction, the values are transformed back to the original price scale.

This improves numerical stability while keeping the final MAE, RMSE, R2, and MAPE interpretable in real price units.

```
[31]: X_train_dl = X_train_processed.to_numpy(dtype=np.float64)  
      X_test_dl = X_test_processed.to_numpy(dtype=np.float64)  
  
      y_train_raw = y_train_np.reshape(-1, 1)  
      y_test_raw = y_test.to_numpy(dtype=np.float64).reshape(-1, 1)  
  
      y_mean = y_train_raw.mean()  
      y_std = y_train_raw.std() + 1e-8  
  
      y_train_dl = (y_train_raw - y_mean) / y_std  
  
      print("X_train:", X_train_dl.shape)  
      print("X_test :", X_test_dl.shape)  
      print("y_train:", y_train_dl.shape)  
      print("Target mean used for scaling:", round(y_mean, 3))
```

```
print("Target std used for scaling :", round(y_std, 3))
```

```
X_train: (107127, 79)
X_test : (26782, 79)
y_train: (107127, 1)
Target mean used for scaling: 470768.601
Target std used for scaling : 163281.344
```

19 20. Step 1 - Implement ReLU

This is directly analogous to the A3 example.

ReLU introduces nonlinearity, which allows the network to learn nonlinear relationships.

Its derivative is needed during backpropagation:

$$\text{ReLU}'(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$

```
[32]: def relu(x):
      return np.maximum(0.0, x)

      def relu_derivative(x):
          return (x > 0).astype(np.float64)
```

20 21. Step 2 - Initialize the Network

Architecture:

input_dim -> 32 -> 16 -> 1

Weights start from small random values.

We use He-style initialization for the ReLU layers:

$$W \sim \text{Normal}(0, 2 / \text{fan_in})$$

Biases start at zero.

The dimensions are:

W1: input_dim x 32
b1: 1 x 32

W2: 32 x 16
b2: 1 x 16

W3: 16 x 1
b3: 1 x 1

```
[33]: input_dim = X_train_dl.shape[1]
hidden1 = 32
hidden2 = 16

rng = np.random.default_rng(SEED)

W1 = rng.normal(
    0,
    np.sqrt(2.0 / input_dim),
    size=(input_dim, hidden1)
)
b1 = np.zeros((1, hidden1))

W2 = rng.normal(
    0,
    np.sqrt(2.0 / hidden1),
    size=(hidden1, hidden2)
)
b2 = np.zeros((1, hidden2))

W3 = rng.normal(
    0,
    np.sqrt(2.0 / hidden2),
    size=(hidden2, 1)
)
b3 = np.zeros((1, 1))

print("W1:", W1.shape, "b1:", b1.shape)
print("W2:", W2.shape, "b2:", b2.shape)
print("W3:", W3.shape, "b3:", b3.shape)
```

```
W1: (79, 32) b1: (1, 32)
W2: (32, 16) b2: (1, 16)
W3: (16, 1) b3: (1, 1)
```

21 22. Step 3 - Forward Propagation

The forward pass computes:

```
Z1 = X @ W1 + b1
H1 = ReLU(Z1)

Z2 = H1 @ W2 + b2
H2 = ReLU(Z2)

Z3 = H2 @ W3 + b3
y_hat = Z3
```

The intermediate values are stored because backpropagation needs them later.

This is the same idea shown in the A3 tutorial, where the `cache` stores the intermediate tensors.

```
[34]: def forward(X):
      z1 = X @ W1 + b1
      h1 = relu(z1)

      z2 = h1 @ W2 + b2
      h2 = relu(z2)

      z3 = h2 @ W3 + b3
      y_hat = z3

      cache = {
          "X": X,
          "z1": z1,
          "h1": h1,
          "z2": z2,
          "h2": h2,
          "z3": z3,
          "y_hat": y_hat,
      }

      return y_hat, cache
```

22 23. Step 4 - Mean Squared Error Loss

For HousePrice regression:

$\text{MSE} = \text{mean}((y_{\text{hat}} - y)^2)$

A small MSE means the predictions are close to the true targets.

The loss function tells the network how wrong its current predictions are.

```
[35]: def mse_loss(y, y_hat):
      return np.mean((y_hat - y) ** 2)
```

23 24. Step 5 - Backpropagation

For MSE:

$dZ3 = 2 * (y_{\text{hat}} - y) / N$

Then:

$dW3 = H2.T @ dZ3$

$db3 = \text{sum}(dZ3)$

$dH2 = dZ3 @ W3.T$

```
dZ2 = dH2 * ReLU'(Z2)
```

```
dW2 = H1.T @ dZ2
```

```
db2 = sum(dZ2)
```

```
dH1 = dZ2 @ W2.T
```

```
dZ1 = dH1 * ReLU'(Z1)
```

```
dW1 = X.T @ dZ1
```

```
db1 = sum(dZ1)
```

This is the same chain-rule idea demonstrated in the supplied A3 material.

Backpropagation is simply the systematic application of the chain rule through the composed functions.

```
[36]: def backward(y, cache):
    Xc = cache["X"]
    z1 = cache["z1"]
    h1 = cache["h1"]
    z2 = cache["z2"]
    h2 = cache["h2"]
    y_hat = cache["y_hat"]

    n = len(Xc)

    # Output layer: linear activation + MSE
    dz3 = 2.0 * (y_hat - y) / n
    dW3 = h2.T @ dz3
    db3 = np.sum(dz3, axis=0, keepdims=True)

    # Hidden layer 2
    dh2 = dz3 @ W3.T
    dz2 = dh2 * relu_derivative(z2)
    dW2 = h1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)

    # Hidden layer 1
    dh1 = dz2 @ W2.T
    dz1 = dh1 * relu_derivative(z1)
    dW1 = Xc.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)

    gradients = {
        "dW1": dW1,
        "db1": db1,
        "dW2": dW2,
        "db2": db2,
```

```

        "dW3": dW3,
        "db3": db3,
    }

    return gradients

```

24 25. Step 6 - Gradient Descent

The parameter update follows:

```

W = W - learning_rate * gradient
b = b - learning_rate * gradient

```

The learning cycle is:

Forward

->

Loss

->

Backward

->

Update

Because this dataset is much larger than the small diabetes example, we use mini-batches. The mathematical operations inside each mini-batch are the same as the full-batch version.

```

[37]: def reset_network(seed=SEED):
        global W1, b1, W2, b2, W3, b3

        local_rng = np.random.default_rng(seed)

        W1 = local_rng.normal(
            0,
            np.sqrt(2.0 / input_dim),
            size=(input_dim, hidden1)
        )
        b1 = np.zeros((1, hidden1))

        W2 = local_rng.normal(
            0,
            np.sqrt(2.0 / hidden1),
            size=(hidden1, hidden2)
        )
        b2 = np.zeros((1, hidden2))

        W3 = local_rng.normal(
            0,
            np.sqrt(2.0 / hidden2),
            size=(hidden2, 1)
        )

```

```

    )
    b3 = np.zeros((1, 1))

reset_network()
print("Network reset completed.")

```

Network reset completed.

25 26. Step 7 - Training Loop

Hyperparameters:

- learning rate = 0.01
- epochs = 40
- batch size = 4096

The loss is recorded after every epoch so that the training process can be visualized.

A decreasing loss curve indicates that the network is learning to reduce its training error.

```

[38]: learning_rate = 0.01
epochs = 40
batch_size = 4096

loss_history = []

num_samples = len(X_train_dl)

training_start = time.time()

for epoch in range(epochs):
    indices = rng.permutation(num_samples)

    epoch_losses = []

    for start_idx in range(0, num_samples, batch_size):
        batch_idx = indices[start_idx:start_idx + batch_size]

        X_batch = X_train_dl[batch_idx]
        y_batch = y_train_dl[batch_idx]

        # Forward
        y_hat_batch, cache = forward(X_batch)

        # Loss
        loss = mse_loss(y_batch, y_hat_batch)
        epoch_losses.append(loss)

```

```

    # Backward
    gradients = backward(y_batch, cache)

    # Gradient descent
    W1 -= learning_rate * gradients["dW1"]
    b1 -= learning_rate * gradients["db1"]

    W2 -= learning_rate * gradients["dW2"]
    b2 -= learning_rate * gradients["db2"]

    W3 -= learning_rate * gradients["dW3"]
    b3 -= learning_rate * gradients["db3"]

    epoch_loss = float(np.mean(epoch_losses))
    loss_history.append(epoch_loss)

    if (epoch + 1) == 1 or (epoch + 1) % 5 == 0:
        print(
            f"Epoch {epoch + 1:02d}/{epochs} | "
            f"MSE = {epoch_loss:.6f}"
        )

dl_train_time = time.time() - training_start

print("Deep Learning training time:", round(dl_train_time, 2), "seconds")

```

```

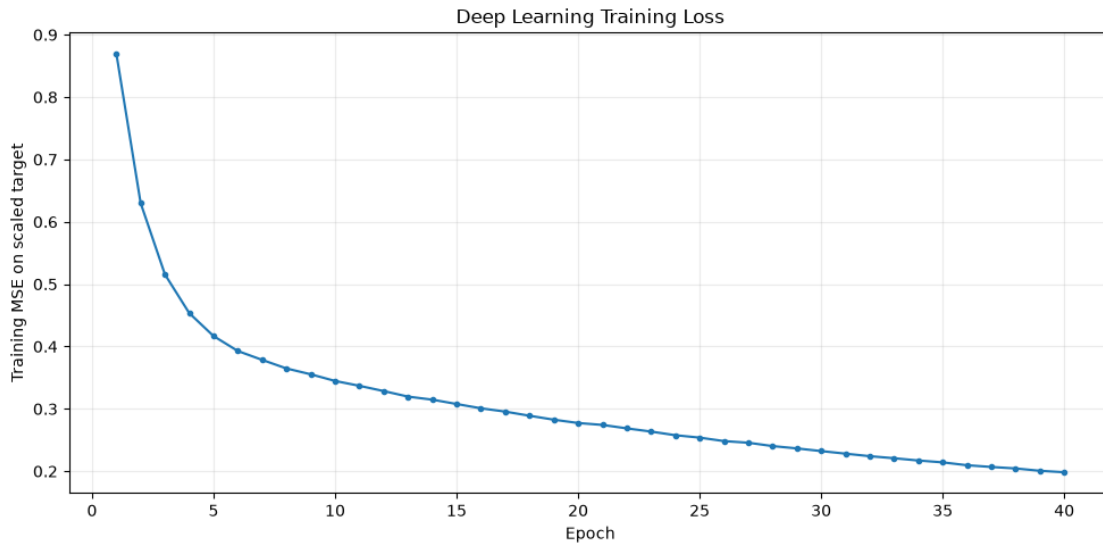
Epoch 01/40 | MSE = 0.869848
Epoch 05/40 | MSE = 0.416728
Epoch 10/40 | MSE = 0.344737
Epoch 15/40 | MSE = 0.307776
Epoch 20/40 | MSE = 0.277279
Epoch 25/40 | MSE = 0.253867
Epoch 30/40 | MSE = 0.232267
Epoch 35/40 | MSE = 0.214180
Epoch 40/40 | MSE = 0.198324
Deep Learning training time: 15.44 seconds

```

```

[39]: plt.figure(figsize=(10, 5))
plt.plot(range(1, epochs + 1), loss_history, marker="o", markersize=3)
plt.title("Deep Learning Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Training MSE on scaled target")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

```



25.0.1 Training curve interpretation

The plot above is analogous to the A3 diabetes challenge that stores `loss_history` and plots epoch versus loss.

Expected behavior:

- early epochs: larger error
- later epochs: lower error
- successful training: overall downward trend

A small amount of fluctuation is normal because mini-batches are shuffled each epoch.

26 27. Step 8 - Deep Learning Prediction

The network predicts the standardized target.

Therefore we convert the prediction back to the original price scale:

```
predicted_price = predicted_scaled * std_train + mean_train
```

This allows the final metrics to be directly compared with the three traditional ML models.

```
[40]: y_pred_dl_scaled, _ = forward(X_test_dl)

y_pred_dl = (
    y_pred_dl_scaled * y_std + y_mean
).reshape(-1)

print("Example Deep Learning predictions:")
display(
```

```

pd.DataFrame({
    "Actual": y_test_raw.reshape(-1)[:10],
    "Predicted": y_pred_dl[:10],
    "Absolute Error": np.abs(
        y_test_raw.reshape(-1)[:10] - y_pred_dl[:10]
    ),
})
)

```

Example Deep Learning predictions:

	Actual	Predicted	Absolute Error
0	235000.0	275818.334490	40818.334490
1	480000.0	490055.939431	10055.939431
2	550000.0	376885.248066	173114.751934
3	498000.0	526427.903121	28427.903121
4	453000.0	517904.473911	64904.473911
5	270000.0	290581.724733	20581.724733
6	610000.0	598570.133526	11429.866474
7	230000.0	265777.199145	35777.199145
8	525000.0	644889.594597	119889.594597
9	838000.0	810018.042268	27981.957732

27 28. Step 9 - Deep Learning Evaluation

The same four regression metrics are now used:

- MAE
- RMSE
- R2
- MAPE

Using the same metrics is essential because it makes the four-model comparison meaningful.

```

[41]: dl_metrics = regression_metrics(y_test_raw.reshape(-1), y_pred_dl)

dl_result = pd.DataFrame([
    {"Model": "Deep Neural Network (NumPy)", **dl_metrics}
])

display(dl_result.round(3))

```

	Model	MAE	RMSE	R2	MAPE (%)
0	Deep Neural Network (NumPy)	53326.421	72788.215	0.798	11.438

```

[42]: plt.figure(figsize=(9, 6))
plt.scatter(
    y_test_raw.reshape(-1),
    y_pred_dl,

```

```

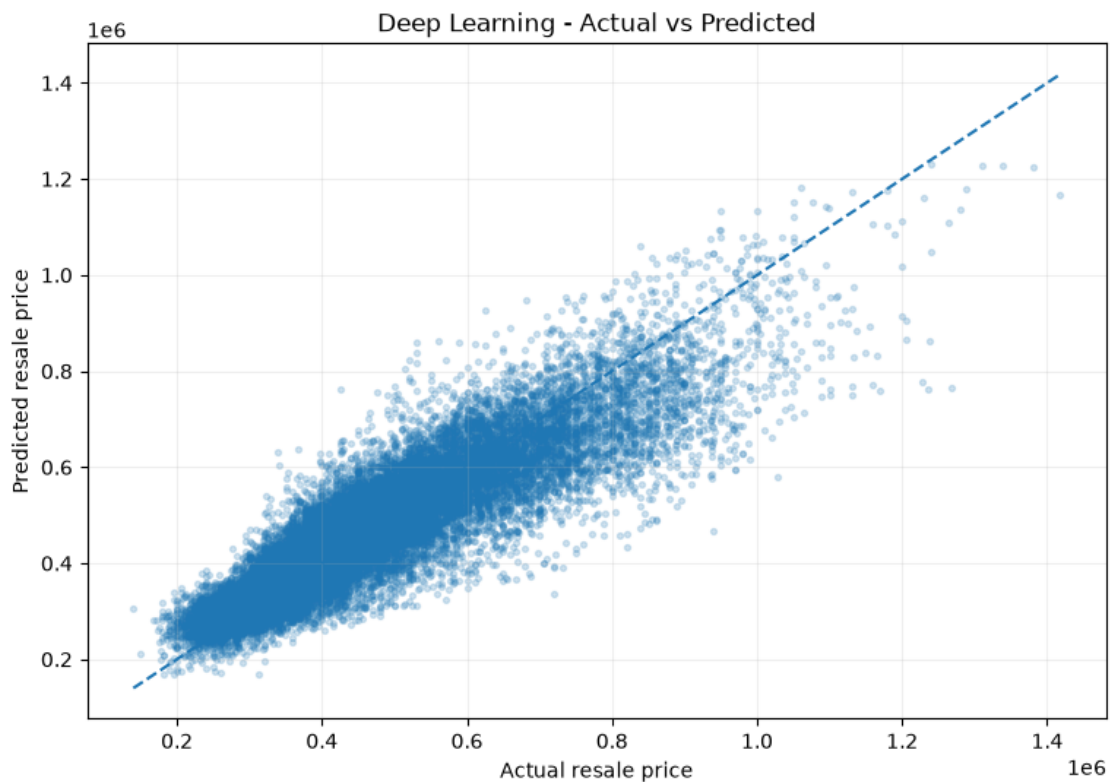
s=8,
alpha=0.2
)

y_min_dl = min(y_test_raw.min(), y_pred_dl.min())
y_max_dl = max(y_test_raw.max(), y_pred_dl.max())

plt.plot(
    [y_min_dl, y_max_dl],
    [y_min_dl, y_max_dl],
    linestyle="--"
)

plt.title("Deep Learning - Actual vs Predicted")
plt.xlabel("Actual resale price")
plt.ylabel("Predicted resale price")
plt.grid(alpha=0.2)
plt.show()

```



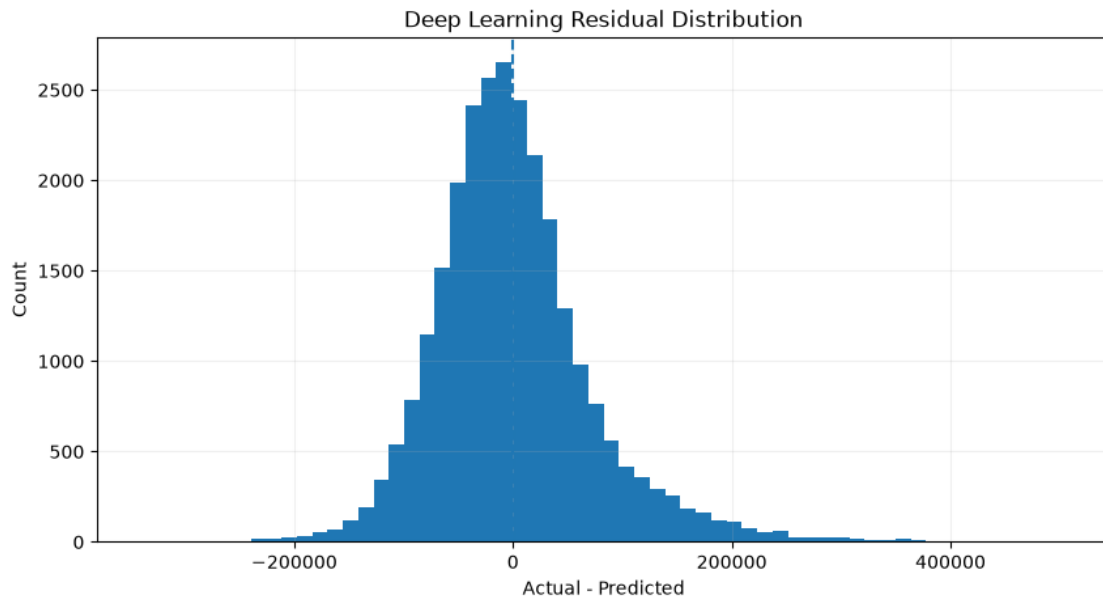
```

[43]: dl_residuals = y_test_raw.reshape(-1) - y_pred_dl

plt.figure(figsize=(10, 5))

```

```
plt.hist(dl_residuals, bins=60)
plt.axvline(0, linestyle="--")
plt.title("Deep Learning Residual Distribution")
plt.xlabel("Actual - Predicted")
plt.ylabel("Count")
plt.grid(alpha=0.2)
plt.show()
```



28 29. Final Comparison - All Four Models Together

This is an important requirement of the assignment.

We do **not**:

compare 3 ML models -> choose one -> compare the winner with Deep Learning

Instead, the correct workflow is:

Model 1

Model 2

Model 3

Model 4

|

v

Evaluate all four on the same test set

|

v

One final comparison table and visualization

All four models are therefore treated as four competing solutions to the same HousePrice regression problem.

```
[44]: all_results = pd.concat(
    [
        traditional_results,
        dl_result
    ],
    ignore_index=True
)

# Rank only for interpretation; all models remain in the same comparison.
all_results["RMSE_rank"] = all_results["RMSE"].rank(method="min")
all_results["MAE_rank"] = all_results["MAE"].rank(method="min")
all_results["R2_rank"] = all_results["R2"].rank(ascending=False, method="min")

display(all_results.sort_values("RMSE").round(3))
```

	Model	MAE	RMSE	R2	MAPE (%)	\
2	HistGradientBoosting	23262.656	31489.940	0.962	5.065	
1	Random Forest	26654.031	37886.863	0.945	5.784	
0	Linear Regression	47471.503	60873.799	0.859	10.817	
3	Deep Neural Network (NumPy)	53326.421	72788.215	0.798	11.438	

	RMSE_rank	MAE_rank	R2_rank
2	1.0	1.0	1.0
1	2.0	2.0	2.0
0	3.0	3.0	3.0
3	4.0	4.0	4.0

28.0.1 How to identify the best model

For a regression task:

- lowest RMSE is better
- lowest MAE is better
- lowest MAPE is better
- highest R2 is better

There may not be one universal winner for every metric. The final conclusion should therefore mention the metric trade-off rather than selecting a model from only one number.

```
[45]: fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

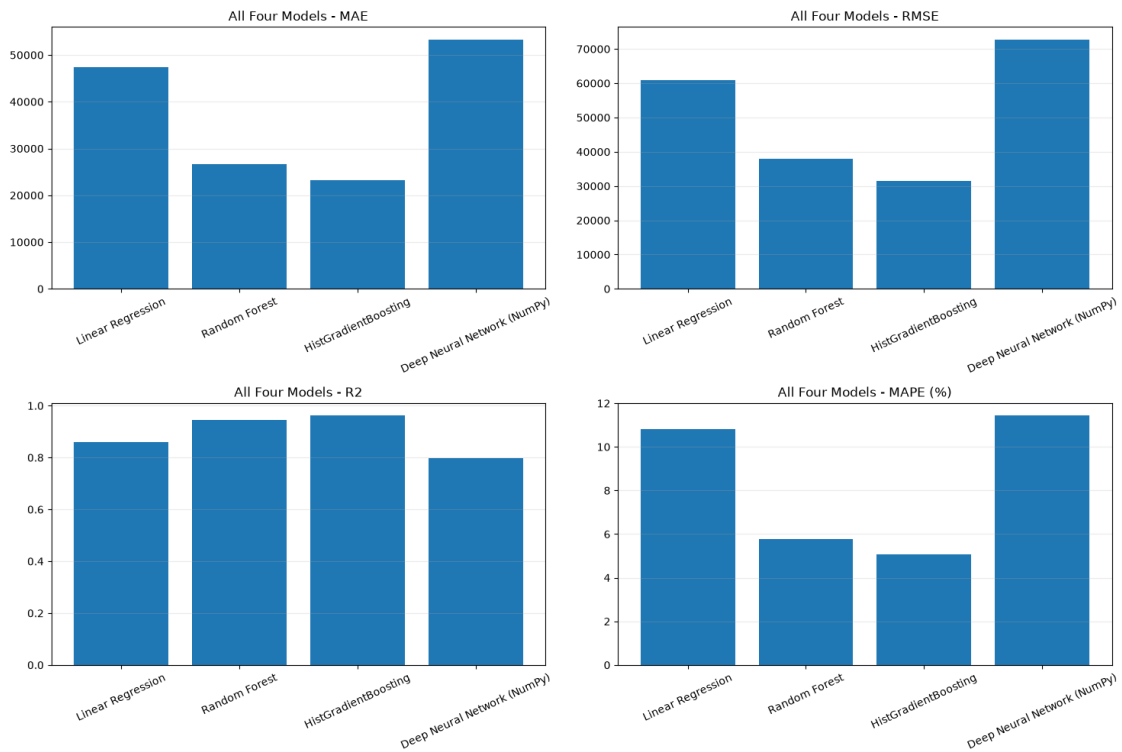
for ax, metric in zip(
    axes,
    ["MAE", "RMSE", "R2", "MAPE (%)"]
):
```

```

values = all_results[metric].values
ax.bar(all_results["Model"], values)
ax.set_title(f"All Four Models - {metric}")
ax.tick_params(axis="x", rotation=25)
ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
plt.show()

```



29 30. Combined Actual vs Predicted Comparison

The following visualization puts all four models into one figure.

A model with predictions concentrated around the diagonal line:

$\text{predicted} = \text{actual}$

is generally performing better.

```

[46]: predictions_all = {
    "Linear Regression": pred_lr,
    "Random Forest": pred_rf,
    "HistGradientBoosting": pred_hgb,
    "Deep Neural Network": y_pred_dl,

```

```

}

plt.figure(figsize=(11, 8))

y_true_plot = np.asarray(y_test)

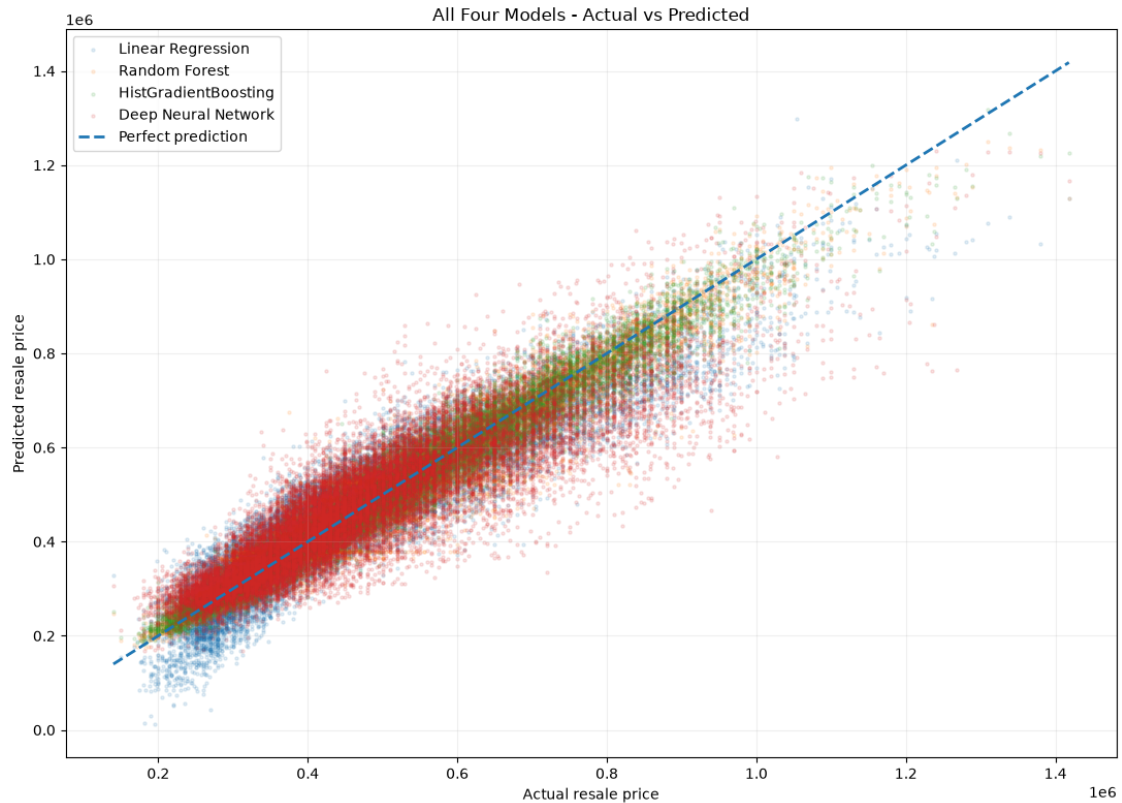
global_min = y_true_plot.min()
global_max = y_true_plot.max()

for name, pred in predictions_all.items():
    plt.scatter(
        y_true_plot,
        pred,
        s=5,
        alpha=0.12,
        label=name
    )

plt.plot(
    [global_min, global_max],
    [global_min, global_max],
    linestyle="--",
    linewidth=2,
    label="Perfect prediction"
)

plt.title("All Four Models - Actual vs Predicted")
plt.xlabel("Actual resale price")
plt.ylabel("Predicted resale price")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()

```



30 31. Residual Comparison - All Four Models

Residual:

$\text{residual} = \text{actual} - \text{predicted}$

A useful model should have residuals centered reasonably close to zero and should avoid extremely large systematic errors.

This visualization helps reveal model behavior that may not be obvious from a single aggregate metric.

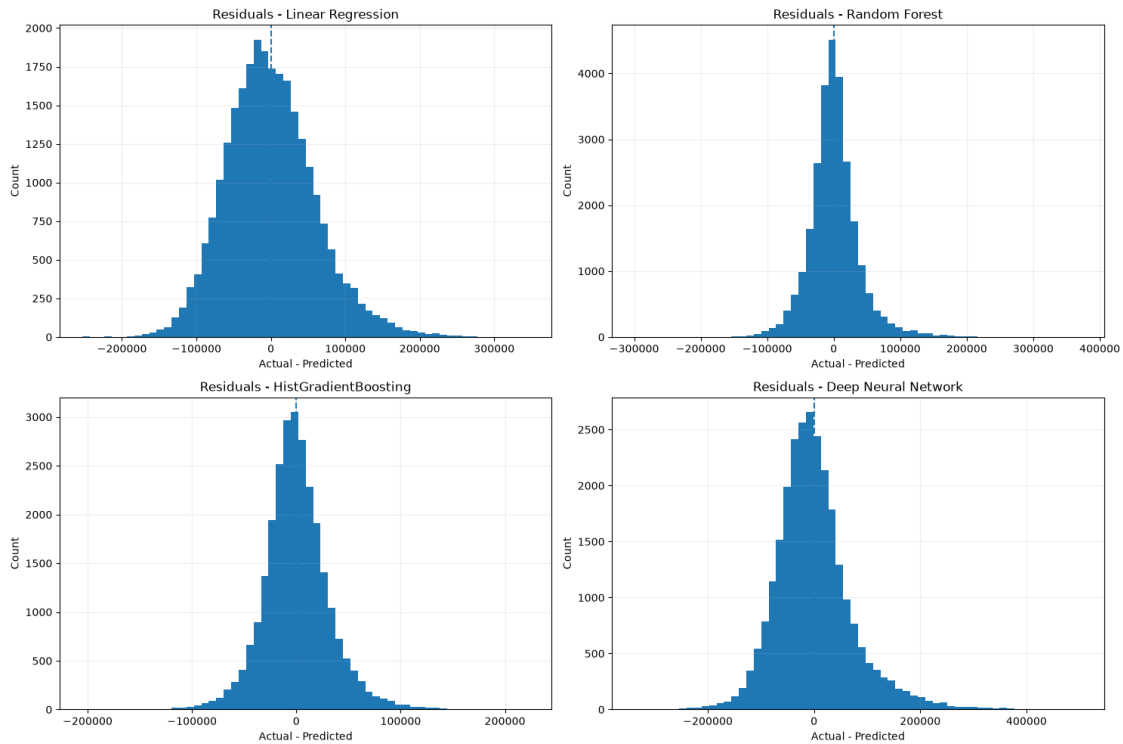
```
[47]: fig, axes = plt.subplots(2, 2, figsize=(15, 10))
      axes = axes.ravel()

      for ax, (name, pred) in zip(axes, predictions_all.items()):
          residuals = y_true_plot - np.asarray(pred)

          ax.hist(residuals, bins=60)
          ax.axvline(0, linestyle="--")
          ax.set_title(f"Residuals - {name}")
          ax.set_xlabel("Actual - Predicted")
```

```
ax.set_ylabel("Count")
ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()
```



31 32. Error Comparison by Price Range

Aggregate metrics can hide where a model performs poorly.

We split the test set into four price quartiles and calculate MAE for each model.

This answers a practical question:

Does a model behave similarly for lower-priced and higher-priced properties?

```
[48]: eval_df = pd.DataFrame({
    "actual": y_true_plot,
    "Linear Regression": pred_lr,
    "Random Forest": pred_rf,
    "HistGradientBoosting": pred_hgb,
    "Deep Neural Network": y_pred_dl,
})
```

```

eval_df["price_group"] = pd.qcut(
    eval_df["actual"],
    q=4,
    labels=["Q1 Low", "Q2", "Q3", "Q4 High"]
)

price_groups = eval_df["price_group"].cat.categories

group_mae = []

for group in price_groups:
    subset = eval_df[eval_df["price_group"] == group]

    for model_name in predictions_all.keys():
        mae_value = mean_absolute_error(
            subset["actual"],
            subset[model_name]
        )

        group_mae.append({
            "Price group": group,
            "Model": model_name,
            "MAE": mae_value
        })

group_mae_df = pd.DataFrame(group_mae)

display(group_mae_df.pivot(
    index="Price group",
    columns="Model",
    values="MAE"
).round(2))

```

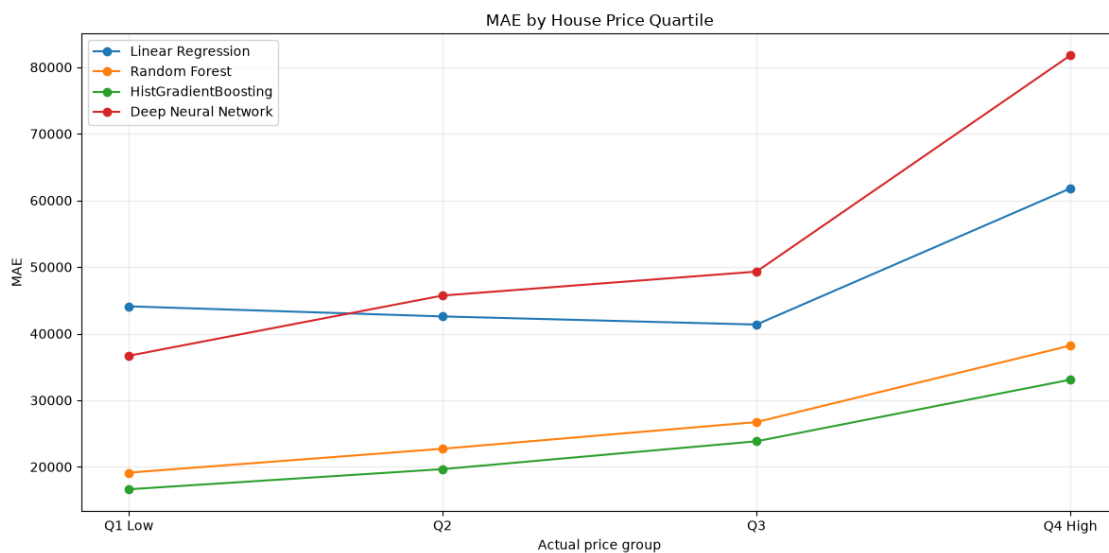
Model	Deep Neural Network	HistGradientBoosting	Linear Regression \
Price group			
Q1 Low	36672.40	16630.69	44097.01
Q2	45715.01	19654.14	42579.62
Q3	49308.26	23835.58	41359.37
Q4 High	81778.41	33086.74	61806.72

Model	Random Forest
Price group	
Q1 Low	19127.70
Q2	22715.97
Q3	26719.74
Q4 High	38206.24

```
[49]: plt.figure(figsize=(12, 6))

for model_name in predictions_all.keys():
    subset = group_mae_df[group_mae_df["Model"] == model_name]
    plt.plot(
        subset["Price group"].astype(str),
        subset["MAE"],
        marker="o",
        label=model_name
    )

plt.title("MAE by House Price Quartile")
plt.xlabel("Actual price group")
plt.ylabel("MAE")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```



32 33. Interpreting the Four Models

32.1 Linear Regression

Strengths:

- simple
- fast
- easy to interpret
- useful baseline

Weakness:

- limited ability to represent nonlinear relationships

32.2 Random Forest

Strengths:

- captures nonlinear patterns
- robust to complex interactions
- does not require a linear relationship

Weakness:

- can require more memory and computation
- may become large with many trees

32.3 HistGradientBoosting

Strengths:

- strong nonlinear regression capability
- sequentially corrects previous errors
- efficient on medium and large tabular datasets

Weakness:

- more hyperparameter-sensitive than a simple linear model

32.4 Deep Neural Network from NumPy

Strengths:

- learns nonlinear transformations through multiple layers
- demonstrates representation learning
- follows the forward -> loss -> backward -> update workflow from the A3 material

Weakness:

- training is implemented manually and is therefore less optimized than modern deep learning libraries
- performance depends strongly on architecture, normalization, learning rate, and number of epochs

33 34. Final Model Ranking

The following cell identifies the best model separately for each metric.

This is only an analytical summary. It does not replace the complete four-model comparison above.

```
[50]: best_rmse = all_results.loc[all_results["RMSE"].idxmin(), ["Model", "RMSE"]]  
best_mae = all_results.loc[all_results["MAE"].idxmin(), ["Model", "MAE"]]  
best_r2 = all_results.loc[all_results["R2"].idxmax(), ["Model", "R2"]]
```

```

best_mape = all_results.loc[all_results["MAPE (%)"].idxmin(), ["Model", "MAPE_
↳(%)"]]

print("Best RMSE :", best_rmse["Model"], "->", round(best_rmse["RMSE"], 3))
print("Best MAE  :", best_mae["Model"], "->", round(best_mae["MAE"], 3))
print("Best R2   :", best_r2["Model"], "->", round(best_r2["R2"], 4))
print("Best MAPE :", best_mape["Model"], "->", round(best_mape["MAPE (%)"], 3),
↳"%")

```

```

Best RMSE : HistGradientBoosting -> 31489.94
Best MAE  : HistGradientBoosting -> 23262.656
Best R2   : HistGradientBoosting -> 0.9623
Best MAPE : HistGradientBoosting -> 5.065 %

```

34 35. A3-Style Deep Learning Checklist

The HousePrice neural network follows the same conceptual sequence as the supplied A3 diabetes example:

34.0.1 Data preparation

Load -> inspect -> clean -> split -> normalize

34.0.2 Network

Input -> Dense -> ReLU -> Dense -> ReLU -> Output

34.0.3 Learning

Forward
-> MSE loss
-> Backpropagation
-> Gradient descent
-> repeat

34.0.4 Evaluation

Prediction
-> MAE
-> RMSE
-> R2
-> MAPE

34.0.5 Visualization

Training loss curve
Actual vs predicted
Residual distribution

Four-model metric comparison
Price-range error comparison

The architecture and loss function are adapted from the A3 diabetes classification example because HousePrice is a regression problem.

35 36. Important Questions and Answers

35.0.1 1. Why is HousePrice a regression problem?

Because `resale_price` is a continuous numerical target.

35.0.2 2. Why are exact duplicates removed?

Exact duplicates repeat the same observation and can give that observation unintended extra weight.

35.0.3 3. How are missing values handled?

For this uploaded CSV, no missing values are present. The general preprocessing rule is median imputation for numeric features and mode imputation for categorical features, using training-set statistics only.

35.0.4 4. Why are statistical outliers kept?

A high-priced property can be a valid real transaction. Removing all IQR outliers could remove useful information from the target distribution.

35.0.5 5. Why is `remaining_lease` converted to a number?

The original text representation is not directly suitable for mathematical models. Converting it to years preserves the underlying numerical information.

35.0.6 6. Why is the test set transformed using training statistics?

Using test-set statistics would allow information from the test set to influence the model pipeline. That is a form of data leakage.

35.0.7 7. Why does the deep learning output layer not use sigmoid?

Sigmoid is suitable for binary classification probabilities. HousePrice is a regression problem, so the final output should be a continuous number.

35.0.8 8. Why is MSE used for the deep learning loss?

MSE is a standard differentiable loss for regression and has a simple derivative for manual back-propagation.

35.0.9 9. What does backpropagation calculate?

It calculates how the loss changes with respect to each trainable weight and bias.

35.0.10 10. What does gradient descent do?

It updates the parameters in the direction that reduces the loss.

35.0.11 11. Are the four models compared fairly?

Yes. They use the same cleaned data, the same train/test split, the same target, and the same evaluation metrics.

35.0.12 12. Which model should be selected?

The final selection should consider all reported metrics and the practical purpose of the task. A model should not be chosen by comparing only one metric.

36 37. Final Conclusion Template

After running all cells, the final conclusion should be written from the actual results in `all_results`.

A suitable conclusion structure is:

The HousePrice dataset was cleaned by removing exact duplicates and checking missing, invalid, and outlier values. No missing values were present in the original dataset, while 259 exact duplicate rows were removed.

The input data was transformed using numeric feature engineering, training-based frequency encoding for high-cardinality categories, one-hot encoding for low-cardinality categories, and training-based standardization.

Three traditional ML models and one Deep Learning model were trained: Linear Regression, Random Forest, HistGradientBoosting, and a NumPy neural network.

All four models were evaluated on the same test set using MAE, RMSE, R2, and MAPE. The final result should identify which model provides the strongest overall balance according to the observed metrics.

The Deep Learning model demonstrates the same fundamental workflow from the A3 tutorial:

Forward -> Loss -> Backpropagation -> Gradient Descent

The neural network was implemented from scratch with NumPy without TensorFlow, PyTorch, Keras, or another Deep Learning framework.

Replace the last two paragraphs with the actual measured results after running the notebook.

37 38. Save the Final Comparison Table

This cell exports the final four-model comparison so it can be reused in the report.

```
[51]: output_csv = notebook_dir / "housePrice_model_comparison.csv"
      all_results.to_csv(output_csv, index=False)

      print("Saved comparison table to:", output_csv.resolve())
```

Saved comparison table to: C:\Users\ADMIN\Downloads\Intelligent
System\assignment 3\DATA\housePrice\housePrice_model_comparison.csv

38 39. Persist the Best ML Model and Deep Learning Model

This section prepares deployment artifacts for the Web and Mobile applications.

The application will load: - the **best traditional ML model** selected from Linear Regression, Random Forest, and HistGradientBoosting; - the **Deep Learning model implemented with NumPy**; - the exact preprocessing information used during training; - metrics and comparison data for visualization.

For this regression problem there is **no predict_proba() output**. Instead, the deployment result contains the predicted price and evaluation metrics. A probability/confidence value should not be invented for a regression model.

```
[52]: import joblib
      from datetime import datetime

      MODEL_DIR = notebook_dir / "model"
      FIG_DIR = MODEL_DIR / "figures"
      MODEL_DIR.mkdir(parents=True, exist_ok=True)
      FIG_DIR.mkdir(parents=True, exist_ok=True)

      # Select ONE best traditional ML model for deployment.
      # RMSE is used as the primary criterion because this is a house-price_
      ↪ regression task.
      best_ml_row = traditional_results.loc[traditional_results["RMSE"].idxmin()]
      best_ml_name = best_ml_row["Model"]

      best_ml_models = {
          "Linear Regression": model_lr,
          "Random Forest": model_rf,
          "HistGradientBoosting": model_hgb,
      }

      best_ml_model = best_ml_models[best_ml_name]

      best_ml_metrics = regression_metrics(y_test_np, best_ml_model.predict(Xte))
```

```

print("Best ML model:", best_ml_name)
display(pd.DataFrame([{
    "Model": best_ml_name,
    **best_ml_metrics
}])).round(3))

```

Best ML model: HistGradientBoosting

	Model	MAE	RMSE	R2	MAPE (%)
0	HistGradientBoosting	23262.656	31489.94	0.962	5.065

```

[53]: # Save the best ML model together with the preprocessing information.
      # This is analogous to diabetes_best_ml_bundle.joblib.

```

```

BEST_ML_PATH = MODEL_DIR / "housePrice_best_ml_bundle.joblib"

best_ml_bundle = {
    "task": "house_price_regression",
    "model_name": best_ml_name,
    "model": best_ml_model,
    "preprocessing_info": preprocessing_info,
    "categorical_low": categorical_low,
    "categorical_high": categorical_high,
    "numeric_features": numeric_features,
    "feature_columns": feature_columns,
    "target_column": target_col,
    "random_seed": SEED,
    "selection_criterion": "RMSE_min",
    "metrics": {
        k: float(v) for k, v in best_ml_metrics.items()
    },
}

joblib.dump(best_ml_bundle, BEST_ML_PATH)

print("Saved best ML bundle:")
print(BEST_ML_PATH.resolve())

```

Saved best ML bundle:

C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\housePrice\model\housePrice_best_ml_bundle.joblib

39 40. Persist the Deep Learning Model

The HousePrice neural network is implemented directly with NumPy, so there is no TensorFlow/Keras/PyTorch model object to serialize.

We therefore save all learned parameters and the target-standardization values in a single **Joblib bundle**. The same preprocessing information is also included so that deployment can reproduce

the exact training representation.

```
[54]: DL_PATH = MODEL_DIR / "housePrice_dl_model.joblib"

dl_bundle = {
    "task": "house_price_regression",
    "model_name": "Deep Neural Network (NumPy)",
    "architecture": {
        "input_dim": int(input_dim),
        "hidden1": int(hidden1),
        "hidden2": int(hidden2),
        "output_dim": 1,
        "activation_hidden": "ReLU",
        "activation_output": "Linear",
        "loss": "Mean Squared Error",
    },
    "weights": {
        "W1": W1.copy(),
        "b1": b1.copy(),
        "W2": W2.copy(),
        "b2": b2.copy(),
        "W3": W3.copy(),
        "b3": b3.copy(),
    },
    "target_scaling": {
        "mean": float(y_mean),
        "std": float(y_std),
    },
    "preprocessing_info": preprocessing_info,
    "categorical_low": categorical_low,
    "categorical_high": categorical_high,
    "numeric_features": numeric_features,
    "feature_columns": feature_columns,
    "target_column": target_col,
    "learning_rate": float(learning_rate),
    "epochs": int(epochs),
    "batch_size": int(batch_size),
    "random_seed": SEED,
    "metrics": {
        k: float(v) for k, v in dl_metrics.items()
    },
}

joblib.dump(dl_bundle, DL_PATH)

print("Saved Deep Learning bundle:")
print(DL_PATH.resolve())
```

Saved Deep Learning bundle:
C:\Users\ADMIN\Downloads\Intelligent System\assignment
3\DATA\housePrice\model\housePrice_dl_model.joblib

40 41. Save Deployment Metrics, Four-Model Comparison, and Visualization

The Web and Mobile applications should not need to run the notebook's training process.

This cell exports: - the selected best ML metrics; - Deep Learning metrics; - all four model comparison metrics; - test predictions from the two deployed models; - deployment-ready visualization images.

These files can be consumed by the backend for dashboards and model comparison.

```
[56]: # Recalculate the two deployment predictions explicitly.
import json
best_ml_pred_deploy = best_ml_model.predict(Xte)
dl_pred_deploy = np.asarray(y_pred_dl).reshape(-1)

# Save metrics in JSON-friendly form.
deployment_metrics = {
    "task": "house_price_regression",
    "target": target_col,
    "best_machine_learning": {
        "model": best_ml_name,
        "metrics": {k: float(v) for k, v in best_ml_metrics.items()},
    },
    "deep_learning": {
        "model": "Deep Neural Network (NumPy)",
        "metrics": {k: float(v) for k, v in dl_metrics.items()},
    },
    "four_model_comparison": all_results.to_dict(orient="records"),
    "probability_output": False,
    "probability_note": (
        "HousePrice is a regression task. Regression models do not produce "
        "class probabilities with predict_proba()."
    ),
    "random_seed": SEED,
}

METADATA_PATH = MODEL_DIR / "housePrice_metadata.json"
METADATA_PATH.write_text(
    json.dumps(deployment_metrics, indent=2),
    encoding="utf-8",
)

# Save test predictions so the backend can reproduce comparison visualizations
```

```

# without retraining.
deployment_predictions = pd.DataFrame({
    "Actual": y_test_np,
    "Best_ML_Predicted": best_ml_pred_deploy,
    "Deep_Learning_Predicted": dl_pred_deploy,
    "Best_ML_Absolute_Error": np.abs(y_test_np - best_ml_pred_deploy),
    "Deep_Learning_Absolute_Error": np.abs(y_test_np - dl_pred_deploy),
})

PREDICTIONS_PATH = MODEL_DIR / "housePrice_deployment_predictions.csv"
deployment_predictions.to_csv(PREDICTIONS_PATH, index=False)

# ---- Deployment visualization 1: metrics ----
plot_df = pd.DataFrame([
    {"Model": best_ml_name, **best_ml_metrics},
    {"Model": "Deep Neural Network (NumPy)", **dl_metrics},
])

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.ravel()

for ax, metric in zip(axes, ["MAE", "RMSE", "R2", "MAPE (%)"]):
    ax.bar(plot_df["Model"], plot_df[metric])
    ax.set_title(f"Deployment Models - {metric}")
    ax.tick_params(axis="x", rotation=20)
    ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
DEPLOY_METRICS_FIG = FIG_DIR / "deployment_models_metrics.png"
plt.savefig(DEPLOY_METRICS_FIG, dpi=180)
plt.show()

# ---- Deployment visualization 2: actual vs predicted ----
plt.figure(figsize=(10, 8))

y_min = min(y_test_np.min(), best_ml_pred_deploy.min(), dl_pred_deploy.min())
y_max = max(y_test_np.max(), best_ml_pred_deploy.max(), dl_pred_deploy.max())

plt.scatter(
    y_test_np,
    best_ml_pred_deploy,
    s=8,
    alpha=0.18,
    label=f"Best ML - {best_ml_name}",
)

plt.scatter(

```

```

    y_test_np,
    dl_pred_deploy,
    s=8,
    alpha=0.18,
    label="Deep Learning - NumPy",
)

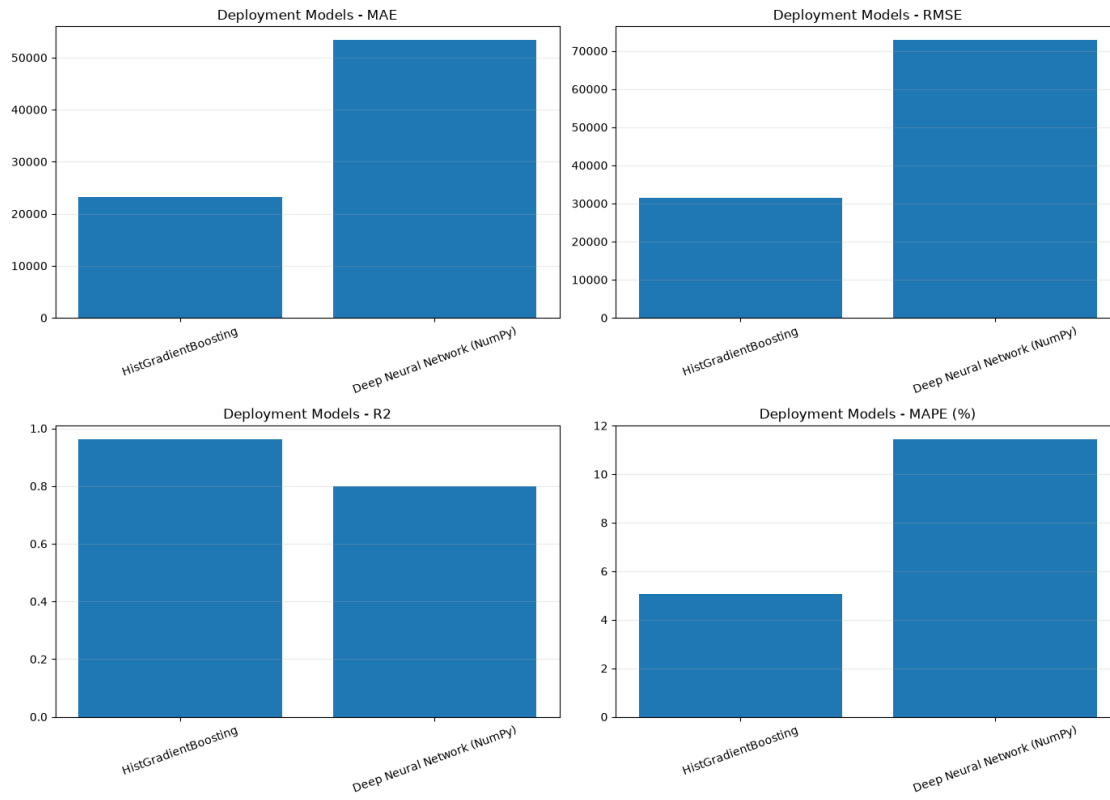
plt.plot(
    [y_min, y_max],
    [y_min, y_max],
    linestyle="--",
    linewidth=2,
    label="Perfect prediction",
)

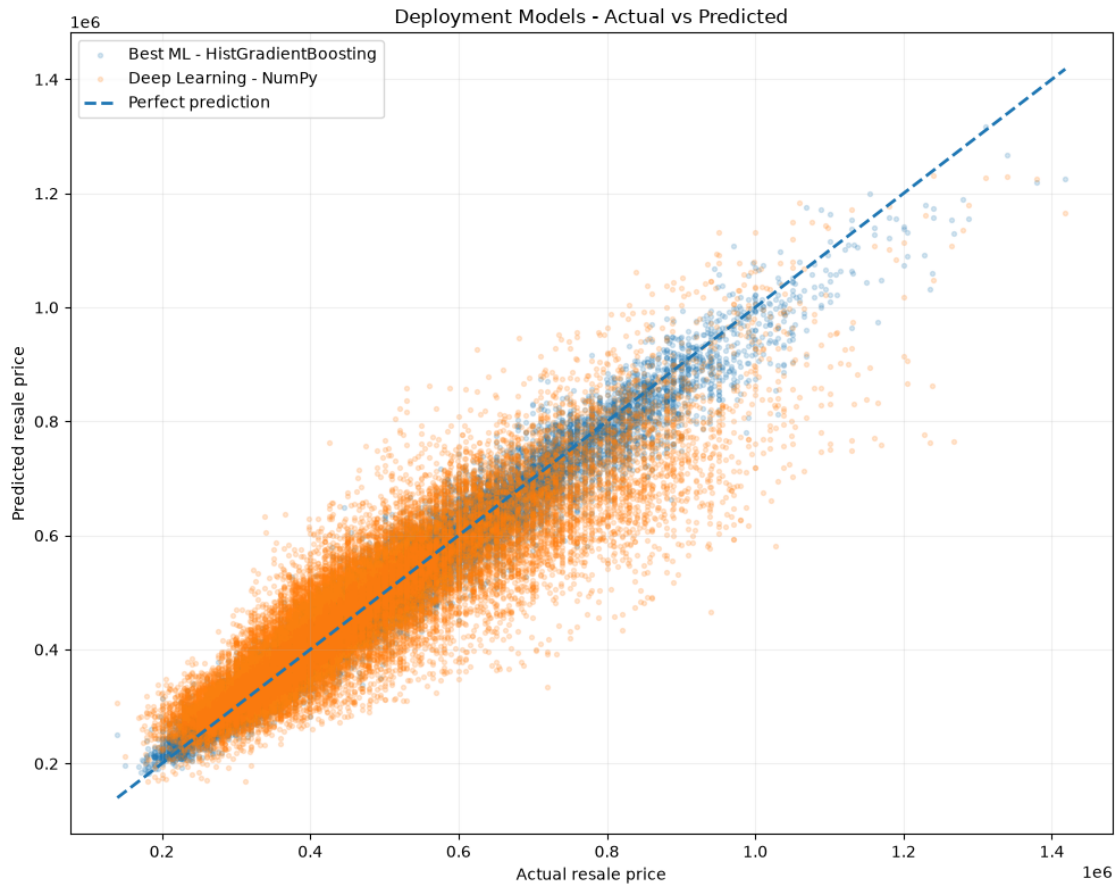
plt.title("Deployment Models - Actual vs Predicted")
plt.xlabel("Actual resale price")
plt.ylabel("Predicted resale price")
plt.legend()
plt.grid(alpha=0.2)
plt.tight_layout()

DEPLOY_SCATTER_FIG = FIG_DIR / "deployment_models_actual_vs_predicted.png"
plt.savefig(DEPLOY_SCATTER_FIG, dpi=180)
plt.show()

print("Saved deployment artifacts:")
for p in [
    BEST_ML_PATH,
    DL_PATH,
    METADATA_PATH,
    PREDICTIONS_PATH,
    DEPLOY_METRICS_FIG,
    DEPLOY_SCATTER_FIG,
]:
    print("-", p.resolve())

```





Saved deployment artifacts:

- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\housePrice_best_ml_bundle.joblib
- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\housePrice_dl_model.joblib
- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\housePrice_metadata.json
- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\housePrice_deployment_predictions.csv
- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\figures\deployment_models_metrics.png
- C:\Users\ADMIN\Downloads\Intelligent System\asssignment 3\DATA\housePrice\model\figures\deployment_models_actual_vs_predicted.png

41 42. Deployment Inference Helper

The following function demonstrates how the application can load the saved Joblib files and return the predictions of both deployed models.

Because the current preprocessing is implemented as explicit functions in the notebook, the saved

preprocessing_info contains the learned medians, category mappings, means, standard deviations, and final feature names. The backend should implement the same transformation logic using these saved parameters and **must not refit them on user input**.

```
[58]: def transform_house_price_input(input_df, info):
    """Apply the exact training-time preprocessing using saved parameters."""
    data = input_df.copy()

    # Numeric imputation
    data[numeric_features] = data[numeric_features].
    ↪ fillna(info["numeric_medians"])

    # High-cardinality frequency encoding
    for col in categorical_high:
        data[col] = data[col].astype(str)
        data[col + "_freq"] = apply_frequency_encoding(
            data[col],
            info["frequency_maps"][col]
        )

    # Low-cardinality categorical imputation + one-hot encoding
    for col in categorical_low:
        data[col] = (
            data[col]
            .fillna(info["categorical_modes"][col])
            .astype(str)
        )

    dummies = pd.get_dummies(
        data[categorical_low],
        prefix=categorical_low,
        dtype=float,
    )
    dummies = dummies.reindex(
        columns=info["dummy_columns"],
        fill_value=0.0,
    )

    cont = pd.concat(
        [
            data[numeric_features].astype(float),
            data[[col + "_freq" for col in categorical_high]].astype(float),
        ],
        axis=1,
    )

    cont = (
```

```

        cont - info["continuous_means"]
    ) / info["continuous_stds"]

    processed = pd.concat(
        [
            cont.reset_index(drop=True),
            dummies.reset_index(drop=True),
        ],
        axis=1,
    )

    return processed[info["feature_names"]].astype(np.float64)

def dl_predict_from_bundle(X_processed, bundle):
    """Run the saved NumPy neural network parameters."""
    weights = bundle["weights"]

    W1_ = weights["W1"]
    b1_ = weights["b1"]
    W2_ = weights["W2"]
    b2_ = weights["b2"]
    W3_ = weights["W3"]
    b3_ = weights["b3"]

    # Convert DataFrame -> NumPy array
    X_array = X_processed.to_numpy(dtype=np.float64)

    h1 = relu(X_array @ W1_ + b1_)
    h2 = relu(h1 @ W2_ + b2_)
    pred_scaled = h2 @ W3_ + b3_

    target_mean = bundle["target_scaling"]["mean"]
    target_std = bundle["target_scaling"]["std"]

    return (pred_scaled * target_std + target_mean).reshape(-1)

# Reload from disk exactly as the application would.
loaded_ml = joblib.load(BEST_ML_PATH)
loaded_dl = joblib.load(DL_PATH)

sample_input = X_test.iloc[[0]].copy()
sample_processed = transform_house_price_input(
    sample_input,
    loaded_ml["preprocessing_info"]
)

```

```

sample_ml_prediction = float(
    loaded_ml["model"].predict(sample_processed)[0]
)

sample_dl_prediction = float(
    dl_predict_from_bundle(sample_processed, loaded_dl)[0]
)

print("Deployment test:")
print("Best ML model:", loaded_ml["model_name"])
print("Best ML prediction:", round(sample_ml_prediction, 2))
print("Deep Learning prediction:", round(sample_dl_prediction, 2))

```

```

Deployment test:
Best ML model: HistGradientBoosting
Best ML prediction: 247916.91
Deep Learning prediction: 275818.33

```

41.1 Deployment artifact structure

After running the new cells, the project can use:

```

model/
  housePrice_best_ml_bundle.joblib
  housePrice_dl_model.joblib
  housePrice_metadata.json
  housePrice_deployment_predictions.csv
figures/
  deployment_models_metrics.png
  deployment_models_actual_vs_predicted.png

```

The Web/Mobile application should load the two .joblib model bundles and use the JSON/CSV/PNG files for metrics and visualization where appropriate.

[]: